



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Studio teorico e numerico di sistemi statistici complessi

Applicazione di metodi Monte Carlo e del Simulated Annealing al *Traveling*

Salesman Problem (TSP)

TESI DI LAUREA TRIENNALE IN
INGEGNERIA FISICA

Autore: **Francesco Savini**

Matricola: 983691
Relatore: Prof. Paolo Biscari
Anno accademico: 2023-24

Abstract

Lo scopo della trattazione è l'analisi del problema TSP mediante l'ausilio di tecniche di tipo Monte Carlo.

A partire da considerazioni di meccanica statistica hamiltoniana, si introduce il tema del campionamento statistico e il suo ruolo nelle simulazioni Monte Carlo a catena di Markov, in particolare nell'algoritmo di Metropolis. Chiariti i concetti di ergodicità e bilancio dettagliato di una catena di Markov, si definiscono la probabilità di selezione e il tasso di accettazione di uno stato nel processo markoviano. A seguire, si mostra come il *Simulated Annealing* (SA), algoritmo di risoluzione di problemi di ottimizzazione combinatoria, sia fondamentalmente un'iterazione dell'algoritmo di Metropolis, e ne sono mostrati i risultati di convergenza principali.

A finire, l'applicazione dell'algoritmo di Metropolis e del *Simulated Annealing* al problema TSP, con particolare attenzione ai differenti esiti delle simulazioni al variare del numero di città, del parametro di controllo e dell'approccio algoritmico usato per la risoluzione. Nel caso del SA si osserva inoltre come il metodo di raffreddamento scelto ha un'influenza cruciale sull'esito della simulazione.

Indice

Abstract	i
Indice	iii
Introduzione	1
1 Richiami di meccanica statistica	3
1.1 Evoluzione hamiltoniana di sistemi statistici e loro dinamica	3
1.2 Ensamble	5
1.3 Equilibrio	6
2 Metodi Monte Carlo a catena di Markov (MCMC)	9
2.1 Stimatori	9
2.2 Campionamento per importanza	9
2.3 Catene di Markov	10
2.4 Ergodicità	13
2.5 Principio del bilancio dettagliato	14
2.6 Tassi di accettazione	15
2.7 Algoritmo di Metropolis	16
2.8 Dalla fisica statistica ai sistemi complessi	17
3 <i>Traveling Salesman Problem</i> (TSP)	19
3.1 Cenni di Teoria della Complessità Computazionale	19
3.2 Algoritmo di Metropolis per il TSP	21
3.2.1 Algoritmo di Metropolis per il TSP: comportamento al variare del parametro di controllo	21
3.2.2 Algoritmo di Metropolis per il TSP: comportamento al variare dello stato iniziale	28
3.2.3 Algoritmo di Metropolis per il TSP: variante a città iniziale variabile	31

3.3	<i>Simulated Annealing per il TSP</i>	34
3.3.1	Schemi di raffreddamento della simulazione e andamento per n piccoli	35
3.3.2	Simulazioni SA con n elevato e schema di raffreddamento geometrico	36
3.3.3	Simulazioni SA con n elevato e schema di raffreddamento logaritmico	40
3.3.4	Conclusioni sul metodo di raffreddamento	42
4	Conclusioni	45
	Bibliografia	47
A	Spazio delle fasi	49
B	Passeggiate aleatorie su grafi	51
	Ringraziamenti	53

Introduzione

Il presente lavoro di tesi trae le sue origini teoriche dallo studio fisico di sistemi statistici a molti componenti. La meccanica statistica si occupa dello studio di tali sistemi, e affronta le difficoltà computazionali legate alla numerosità dei costituenti attuando una trattazione statistica delle proprietà macroscopiche del sistema. In altre parole, la fisica statistica stabilisce il ponte fra i fenomeni termici macroscopici e il mondo microscopico, senza stabilire una descrizione microscopicamente dettagliata del problema. Nell'ambito della meccanica statistica, è cruciale il confronto fra i valori misurati di talune grandezze fisiche ed il valore teorico che per esse è possibile determinare, ovvero il loro valore di aspettazione. Per effettuare il calcolo di valori di aspettazione in modo approssimato è importante sviluppare tecniche di campionamento degli stati del sistema, ossia metodi per scegliere quali stati del sistema includere (fra tutti quelli possibili nello spazio delle configurazioni) per ottenere una buona stima del valore d'aspettazione dell'osservabile in questione. Il campionamento per importanza (*importance sampling*) è un concetto alla base dei metodi Monte Carlo. Questi ultimi sono descrivibili come metodi computazionali utilizzati per la risoluzione di problemi complessi (anche se completamente deterministici): mediante il campionamento statistico si seleziona un numero ristretto di stati del sistema e si effettuano calcoli stocastici per ottenere risultati numerici approssimati. L'esempio più comune di implementazione di metodi Monte Carlo è rappresentato dall'algoritmo di Metropolis. I metodi Monte Carlo, utilizzati in tutte le discipline nelle quali sono richieste grandi risorse computazionali, sono alla base di alcuni algoritmi utilizzati per la risoluzione di problemi di ottimizzazione combinatoria, fra cui il *Simulated Annealing*. Nel presente lavoro, l'algoritmo di Metropolis e il *Simulated Annealing* saranno utilizzati per lo studio del prototipo di problema di ottimizzazione combinatoria: il problema del commesso viaggiatore (*traveling salesman problem*, TSP). Quest'ultimo è il più celebre esempio di problema nella teoria della complessità computazionale, ed è ampiamente studiato nella ricerca operativa.

1 | Richiami di meccanica statistica

1.1. Evoluzione hamiltoniana di sistemi statistici e loro dinamica

La meccanica statistica è il settore della fisica che si occupa dello studio di sistemi con numero elevato di elementi, dell'ordine del numero di Avogadro ($N_A = 6.02214076 \cdot 10^{23} \text{ mol}^{-1}$), che non potrebbero essere analizzati senza l'ausilio di strumenti statistici. Ogni componente *i-esima* del sistema, tipicamente un atomo o una molecola, è caratterizzata da una **coordinata generalizzata** q_i e da un **momento generalizzato** p_i . Il sistema è detto *hamiltoniano* se rispetta le equazioni di Hamilton:

$$\dot{q}_i = \frac{\partial H}{\partial p_i} \quad (1.1)$$

$$\dot{p}_i = -\frac{\partial H}{\partial q_i} \quad (1.2)$$

dove H è la funzione *hamiltoniana* del sistema. Si ricorda che, in un sistema di N elementi caratterizzato da energia potenziale $U(\mathbf{q}, t)$ e da energia cinetica totale $T(\mathbf{q}, \dot{\mathbf{q}})$, si definisce la funzione **lagrangiana**:

$$L(\mathbf{q}, \dot{\mathbf{q}}, t) := T(\mathbf{q}, \dot{\mathbf{q}}) - U(\mathbf{q}, t) \quad (1.3)$$

La funzione **hamiltoniana** del sistema è definita come la *trasformata di Legendre* della lagrangiana:

$$H(\mathbf{q}, \mathbf{p}, t) := \sum_{i=1}^N p_i \dot{q}_i - L(\mathbf{q}, \dot{\mathbf{q}}) \quad (1.4)$$

ed il momento p_i coniugato alla coordinata generalizzata q_i si definisce come:

$$p_i = \frac{\partial L}{\partial \dot{q}_i} \quad (1.5)$$

Le equazioni (1.1) e (1.2) definiscono un sistema di $6N$ equazioni differenziali alle derivate parziali, dove N è il numero di costituenti del sistema.

Se il sistema hamiltoniano fosse isolato, l'energia si conserverebbe. Quindi, noto lo spettro dei livelli energetici, sia esso discreto $\{E_1, E_2, \dots\}$ o continuo, il sistema isolato rimarrebbe nello stesso stato ad energia $\bar{E}$, o al più transirebbe fra stati degeneri.

Un'altra possibile e più interessante situazione è la seguente: il sistema di N componenti si trova in contatto termico con un termostato (*bagno termico*), detto anche *reservoir*, di $N_r \gg N$ costituenti, caratterizzato da capacità termica molto elevata: $C_r \rightarrow \infty$. In questo caso, in generale, l'energia del sistema non si conserva a causa degli scambi termici con il termostato, e a rimanere costanti sono solo N e la temperatura T , quest'ultima evidentemente uguale a quella del *reservoir* T_r . Il termostato può essere introdotto come perturbazione all'hamiltoniana del sistema, e, in questa prospettiva, si definisce una **dinamica**, cioè un insieme di *probabilità di transizione per unità di tempo*, o *rate di transizione*. Siano μ e ν due generici stati del sistema, e sia $R_{\mu\nu}$ il rate di transizione da μ a ν : la probabilità della transizione $\mu \rightarrow \nu$ in un tempo dt vale allora $R_{\mu\nu}dt$. Generalmente si suppone che i rate, definiti per ogni coppia di stati del sistema, siano tempo-indipendenti. Sia ora $w_\mu(t)$ la probabilità che, al tempo t , il sistema si trovi nello stato μ . Vale allora il seguente sistema di equazioni (**master equations**):

$$\frac{dw_\mu}{dt} = \sum_\nu [w_\nu(t)R_{\nu\mu} - w_\mu(t)R_{\mu\nu}], \forall \text{ stato } \mu \quad (1.6)$$

con il vincolo triviale:

$$\sum_\mu w_\mu(t) = 1, \forall t, \mu \quad (1.7)$$

In generale, per un sistema ad elevato numero di costituenti, la risoluzione delle equazioni di Hamilton è computazionalmente irrealizzabile; si sceglie quindi di descrivere il sistema mediante l'ausilio di grandezze macroscopiche (come pressione e temperatura nel caso di sistemi termodinamici elementari). Tali grandezze macroscopiche definiscono un **macro-stato** del sistema; ad ogni macrostato corrisponde un gran numero di **microstati**, questi ultimi individuati dai valori di tutte le coordinate e dei momenti generalizzati dei costituenti, cioè da punti specifici nello *spazio delle configurazioni* Γ del sistema. Si definisce **molteplicità** di un macrostato il numero di microstati con cui è possibile realizzarlo.

Una qualsiasi grandezza fisica Q del sistema è in generale una funzione delle coordina-

te e dei momenti generalizzati. Dato il set di probabilità $\{w_\mu\}$, si definisce il valore di aspettazione di tale grandezza fisica.

Definizione 1.1. *Il valore di aspettazione della grandezza fisica Q è dato da:*

$$\langle Q \rangle := \sum_{\mu} Q_{\mu} w_{\mu}(t), \quad (1.8)$$

dove Q_{μ} è il valore assunto da Q nello stato μ .

Nell'**atto di misura** dell'osservabile Q con una qualsivoglia procedura sperimentale, tuttavia, i tempi di misurazione sono tipicamente molto maggiori dei tempi tipici di evoluzione dinamica dei singoli componenti del sistema; di conseguenza, ciò che viene effettivamente misurato non è il valore *istantaneo* di Q , bensì la sua **media temporale** nell'intervallo di misurazione $[0, \tau]$:

$$\langle Q \rangle_t := \frac{1}{\tau} \int_0^{\tau} Q(\mathbf{q}(t), \mathbf{p}(t), t) dt \quad (1.9)$$

1.2. Ensemble

La difficoltà nel confronto fra risultato sperimentale e previsione teorica, insita nella concreta incalcolabilità di $\langle Q \rangle_t$, viene superata mediante il concetto di **ensemble** (introdotto da Gibbs nel 1878). Per ensemble si intende un insieme di copie del sistema in questione, ciascuna interagente con il proprio termostato, che possono differire solo per i valori di coordinate e momenti $\{q_i(t), p_i(t)\}$, tutte però corrispondenti allo stesso macrostato, e descritte quindi da valori uguali di grandezze macroscopiche quali pressione e temperatura. L'ensemble corrisponde quindi ad un insieme di punti nello spazio delle fasi Γ , ed è caratterizzato da una certa evoluzione dinamica (si parla di *moto dello sciame*) in Γ . Un punto in Γ che rappresenta lo stato di un sistema ad N particelle è detto *punto rappresentativo*.

Matematicamente, l'ensemble si definisce come una **misura di probabilità** nello spazio delle fasi:

$$\rho : \Gamma \times I_t \rightarrow \mathbb{R}^+,$$

Da cui segue la condizione di normalizzazione della misura:

$$\int_{\Gamma} \rho(\mathbf{q}(t), \mathbf{p}(t), t) d\mathbf{q}d\mathbf{p} = 1 \quad (1.10)$$

Vale il seguente risultato, dovuto a Liouville:

Teorema 1.1. *Sia $\rho(\mathbf{q}, \mathbf{p}, t)$ un ensemble di un sistema hamiltoniano. Vale allora:*

$$\frac{d\rho}{dt} = \frac{\partial \rho}{\partial t} + \sum_{i=1}^{3N} \left(\frac{\partial \rho}{\partial p_i} \dot{p}_i + \frac{\partial \rho}{\partial q_i} \dot{q}_i \right) = 0 \quad (1.11)$$

L'interpretazione del teorema di Liouville è semplice da immaginare. Immaginando di seguire il moto di un punto rappresentativo in Γ , la densità di punti rappresentativi nel suo intorno è costante. La distribuzione di punti rappresentativi si muove quindi in Γ in modo analogo a un fluido incompressibile nello spazio.

Un osservabile (grandezza fisica) Q è un'applicazione: $Q : \Gamma \rightarrow \mathbb{R}$.

Definizione 1.2. *La **media sull'ensemble** dell'osservabile Q è data dall'espressione:*

$$\langle Q \rangle_e := \int_{\Gamma} \rho(\mathbf{q}, \mathbf{p}, t) Q(\mathbf{q}(t), \mathbf{p}(t), t) d\mathbf{q} d\mathbf{p} \quad (1.12)$$

Le due diverse definizioni di media di Q (media sull'ensemble e media temporale) coincidono solo nell'ipotesi in cui nell'intervallo $[0, \tau]$ il sistema passi per un *insieme di stati rappresentativo dell'intero spazio delle configurazioni* (**ipotesi ergodica**). Questa condizione, assolutamente non scontata da verificare, verrà ripresa nella sezione 2.4 sull'ergodicità.

1.3. Equilibrio

Uno stato del sistema si dice **di equilibrio** se:

$$\frac{dw_\mu}{dt} = 0, \forall \text{ stato } \mu \quad (1.13)$$

Dalla (1.6), si nota che, posto il vincolo $w_\mu \in [0, 1]$ (che esclude nella (1.13) soluzioni esponenzialmente divergenti), il sistema che rispetta le *master equations* raggiungerà prima o poi un equilibrio, sia esso semplice o dinamico (ciclo limite). Si definiscono le **probabilità di occupazione di equilibrio**:

$$p_\mu := \lim_{t \rightarrow \infty} w_\mu(t) \quad (1.14)$$

Si può dimostrare (*Gibbs*) che, per un sistema all'equilibrio termico con il termostato a temperatura T , vale:

$$p_\mu = \frac{1}{Z} e^{-\frac{E_\mu}{k_B T}}, \quad (1.15)$$

dove k_B è la costante di Boltzmann, e si definisce il parametro $\beta := \frac{1}{k_B T}$. Z è detta **funzione di partizione**, e garantisce la normalizzazione della distribuzione di probabilità:

$$Z := \sum_{\mu} e^{-\beta E_\mu} \quad (1.16)$$

All'equilibrio, quindi, il valore di aspettazione dell'osservabile Q vale:

$$\langle Q \rangle = \frac{1}{Z} \sum_{\mu} Q_\mu e^{-\beta E_\mu} \quad (1.17)$$

La funzione partizione Z riassume in sé tutte le proprietà del sistema. Più precisamente, la risposta del sistema ad una generica perturbazione di una grandezza macroscopica (come il volume) è calcolabile mediante relazioni che coinvolgono Z e la variabile termodinamica coniugata a quella perturbata (in questo caso la pressione).

Inoltre, si possono ricavare le seguenti relazioni:

$$U = -\frac{\partial \log Z}{\partial \beta} \text{ (energia interna)} \quad (1.18)$$

$$S = -k_B \beta \frac{\partial \log Z}{\partial \beta} \text{ (entropia)} \quad (1.19)$$

$$F = U - TS = -k_B T \log Z \text{ (energia libera di Helmholtz)} \quad (1.20)$$

2 | Metodi Monte Carlo a catena di Markov (MCMC)

2.1. Stimatori

Il calcolo di Z è estremamente complesso tranne che per sistemi particolarmente semplici (come il gas perfetto). **Lo scopo dei metodi Monte Carlo in meccanica statistica è calcolare valori di aspettazione di grandezze fisiche senza passare per il calcolo della funzione partizione.** L'idea è quella di poter approssimare $\langle Q \rangle$ con una somma su un sottoinsieme di stati $D := \{\mu_1, \dots, \mu_M\}$. I metodi Monte Carlo selezionano un sottoinsieme di stati per mezzo di una distribuzione di probabilità p_μ data. Tale operazione è detta **campionamento**. Si definisce **stimatore** di Q la quantità:

$$\hat{Q}_M = \frac{\sum_{\mu \in D} Q_\mu p_\mu^{-1} e^{-\beta E_\mu}}{\sum_{\mu \in D} p_\mu^{-1} e^{-\beta E_\mu}} \quad (2.1)$$

Se lo stimatore è costruito in modo da essere *non distorto* e *consistente in media quadratica* (vedi [10]) vale:

$$\lim_{M \rightarrow \infty} \hat{Q}_M = \langle Q \rangle \quad (2.2)$$

La scelta della distribuzione p_μ è alla base del concetto di **campionamento per importanza** (*importance sampling*).

2.2. Campionamento per importanza

Scegliendo la distribuzione p_μ uniforme $p_\mu \sim \mathcal{U}$, si otterrebbe:

$$\hat{Q}_M = \frac{\sum_{\mu \in D} Q_\mu e^{-\beta E_\mu}}{\sum_{\mu \in D} e^{-\beta E_\mu}} \quad (2.3)$$

Si arguisce immediatamente che, per il calcolo approssimato di $\langle Q \rangle$, tale scelta è scorretta. Un sistema fisico a temperatura T del *reservoir*, infatti, esplora nella sua naturale evoluzione un numero limitato di stati, caratterizzati da diversi valori di energia. Si dimostra che le fluttuazioni dell'energia seguono la relazione approssimata:

$$\frac{\delta E}{E} \approx \frac{1}{\sqrt{N}}, \quad (2.4)$$

dove N è il numero di costituenti. Per N elevato, tali fluttuazioni sono piccole, e il numero di stati che contribuisce al valore di aspettazione è quindi molto ridotto. Per tenere conto di ciò, si utilizzano altri andamenti per p_μ , e non quello uniforme. Se si considera l'evoluzione di un sistema fisico qualsiasi (come un recipiente contenente gas) sulle scale temporali tipiche di un esperimento in laboratorio, si nota che il sistema passa in tale intervallo di tempo per un numero di stati *molto minore* del numero totale di (micro)stati per esso disponibili (vedi appendice A). Si può affermare che, in natura e su scale temporali modeste, i sistemi osservati effettuano una spontanea simulazione Monte Carlo, selezionando un numero limitato di stati fra quelli occupabili. Il sistema, naturalmente, non campiona gli stati mediante una distribuzione uniforme, bensì utilizza la distribuzione di Boltzmann (1.15). Da ciò segue che, nelle nostre simulazioni, può essere una buona scelta utilizzare tale distribuzione per campionare gli stati:

$$p_\mu = \frac{1}{Z} e^{-\beta E_\mu} \Rightarrow \hat{Q} = \frac{1}{M} \sum_{j=1}^M Q_{\mu_j} \quad (2.5)$$

2.3. Catene di Markov

All'interno di questo schema, il compito di selezionare un sottosinsieme di stati mediante la distribuzione di Boltzmann è affidato ai **processi Markoviani**. I metodi Monte Carlo che fanno uso di catene di Markov sono detti **Metodi Monte Carlo a catena di Markov (MCMC)**. Un processo di Markov può essere considerato come un meccanismo che, a partire da uno stato μ del sistema, fornisce un altro stato ν . Tale meccanismo di proposizione di un nuovo stato a partire da uno presente è **stocastico** (cioè, a partire dallo stesso stato μ , possono essere generati stati diversi ν con probabilità $P_{\mu\nu}$) e **unicamente dipendente dallo stato μ di partenza**. Introduciamo adesso alcuni concetti cruciali per la comprensione dei processi markoviani. In particolare, ci soffermeremo sulle **catene di Markov tempo-discrete (DTMC)**.

Definizione 2.1. Una *catena di Markov tempo-discreta (DTMC)* è una sequenza

di variabili aleatorie $\{X_1, \dots, X_M\}$ sullo spazio degli eventi $\Gamma = \{x_1, \dots, x_M\}$ tali che:

$$\forall x, y \in \Gamma, \forall t > 1, \forall \text{ evento } H_{t-1} = \bigcap_{s=1}^{t-1} \{X_s = x_s\} \text{ che soddisfa } P(H_{t-1} \cap \{X_t = x\}) > 0,$$

si ha:

$$P(X_{t+1} = y | H_{t-1} \cap X_t = x) = P(X_{t+1} = y | X_t = x) \text{ (*proprietà di Markov*)} \quad (2.6)$$

La distribuzione di probabilità di ogni variabile aleatoria della catena è quindi unicamente dipendente da quella della precedente.

OSSERVAZIONE: Nella (2.6) non è presente una dipendenza della probabilità di transizione dal tempo. Nella presente trattazione prenderemo questo aspetto come parte integrante della definizione di catena di Markov. Altri autori, tuttavia, riportano questo come un caso particolare di una definizione più generale di processo markoviano, nella quale le probabilità di transizione dipendono esplicitamente dal tempo. Seguendo questo *modus operandi*, una catena di Markov come quella definita precedentemente è detta **omogenea**.

La proprietà di Markov si può riassumere affermando che il sistema è *senza memoria*, ossia le probabilità di transizione non dipendono dagli stati precedentemente occupati dal sistema, ma solo da quello corrente e da quello verso cui avviene la transizione. Nel nostro caso, lo spazio degli eventi coincide con lo spazio delle fasi Γ del sistema, cioè con l'insieme degli stati μ accessibili dal sistema. Si definisce **matrice stocastica** del processo di Markov la matrice $\mathbf{P}$ che raccoglie le probabilità di transizione $P_{\mu\nu}$. È evidente pertanto la seguente:

Proposizione 2.1. *La somma delle probabilità su una riga della matrice stocastica vale 1:*

$$\sum_{\nu} P_{\mu\nu} = 1 \quad (2.7)$$

Ossia, il processo di Markov genera sicuramente un qualche stato ν a partire da μ . Vale inoltre:

Proposizione 2.2. *Una matrice stocastica ha almeno un autovalore 1 con autovettore $[1, 1, \dots, 1]^T$, con $\dim([1, 1, \dots, 1]^T) = \text{card}(\Gamma)$.*

Questa proprietà si evince facilmente dalla (2.7).

DIMOSTRAZIONE. Sia $\mathbf{v} = [1, \dots, 1]^T$ un vettore di uni di dimensione uguale al numero di stati. Sia inoltre $\mathbf{P}$ una matrice stocastica, che gode quindi delle proprietà seguenti:

- $P_{ij} \geq 0, \forall i, j$;
- $\sum_j P_{ij} = 1$.

Allora:

$$(\mathbf{P} \cdot \mathbf{v})_i = \sum_j P_{ij} = 1, \forall i \quad (2.8)$$

Segue allora la tesi: $\mathbf{P} \cdot \mathbf{v} = \mathbf{v}$

Definizione 2.2. Un vettore stazionario di probabilità π è un autovettore di $\mathbf{P}$, con autovalore 1:

$$\mathbf{P} \cdot \pi = \pi \quad (2.9)$$

Definizione 2.3. Una catena di Markov si dice **irriducibile** se, per ogni coppia (μ, ν) di stati, la probabilità di raggiungere ν da μ è diversa da 0, e viceversa:

$$\forall \mu, \nu, \exists m, n \in \mathbb{N} : P_{\mu\nu}^n > 0 \wedge P_{\nu\mu}^m > 0, \quad (2.10)$$

dove $P_{\mu\nu}^n$ e $P_{\nu\mu}^m$ sono rispettivamente la probabilità di raggiungere ν da μ in n passi e quella di raggiungere μ da ν in m passi.

Si dice che i due stati μ, ν **appartengono alla stessa classe**; una catena di Markov è quindi irriducibile se tutti gli stati appartengono alla stessa classe.

Definizione 2.4. Sia $T(\mu) := \{t \geq 1 : P_{\mu\mu}^t > 0\}$ l'insieme dei tempi per cui è possibile per la catena ritornare allo stato μ di partenza. Il **periodo** dello stato μ è il massimo comun divisore di $T(\mu)$.

Definizione 2.5. Una catena di Markov si dice **aperiodica** se tutti gli stati hanno periodo 1.

Sia ora $f_i(n)$ la **probabilità di primo ritorno** ad uno stato μ , in cui la grandezza X vale i :

$$f_i(n) = P(X_n = i | X_0 = i, X_1 \neq i, \dots, X_{n-1} \neq i),$$

e f_i^* la corrispondente probabilità di ritorno ad uno step qualsiasi:

$$f_i^* := \sum_{n=1}^{\infty} f_i(n) \quad (2.11)$$

Definizione 2.6. Uno stato μ di una catena di Markov si dice ***persistente*** (o *ricorrente*) se $f_i^* = 1$, altrimenti si dice ***transiente***.

Nelle simulazioni Monte Carlo, si utilizzano processi di Markov per generare sequenze di stati in modo tale che, partendo da un qualsiasi stato, gli stati generati compaiano (per tempi sufficientemente lunghi) con probabilità date dalla distribuzione di Boltzmann.

N.B.: Le definizioni e le classificazioni per le catene di Markov descritte in questo paragrafo sono alla base della formulazione dei teoremi di convergenza per l'algoritmo di Simulated Annealing (contenuti in [2]), che verrà utilizzato nella sezione 3.3 per risolvere il TSP.

2.4. Ergodicità

Stabilire l'ergodicità di un sistema fisico vuol dire constatare che, per tempi lunghi, il sistema esplora tutta la zona dello spazio delle fasi compatibile con i vincoli cui è soggetto. In altri termini:

Definizione 2.7. Un ensemble $\rho(\mathbf{p}, \mathbf{q}, t)$ si dice ***ergodico*** se le medie temporali e sull'ensemble coincidono per tempi lunghi:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T Q(\mathbf{q}(t), \mathbf{p}(t), t) dt = \int_{\Gamma} \rho(\mathbf{q}, \mathbf{p}, t) Q(\mathbf{q}(t), \mathbf{p}(t), t) d\mathbf{q} d\mathbf{p} \quad (2.12)$$

Similmente, l'ergodicità del processo di Markov utilizzato nella simulazione Monte Carlo equivale ad affermare che, da ogni stato μ è possibile raggiungere mediante un numero finito di passi un qualsiasi altro stato ν . Richiediamo l'ergodicità del processo di Markov, cioè l'esistenza di almeno un percorso che collega due stati qualsiasi μ e ν , $\forall \mu, \nu$, in quanto, nel sistema fisico all'equilibrio, ogni stato ha una probabilità non nulla (vedi (1.15)) di essere occupato.

Definiamo ora in termini più rigorosi il concetto di ergodicità per una catena di Markov.

Definizione 2.8. Uno stato di una DTMC si dice ***ergodico*** se è persistente e aperiodico.

Definizione 2.9. Una catena di Markov tempo-discreta si definisce ***ergodica*** se ogni stato è ergodico.

2.5. Principio del bilancio dettagliato

Imporre la condizione di bilancio dettagliato è cruciale per assicurarsi che il processo di Markov arrivi (all'equilibrio) a generare stati secondo la distribuzione di Boltzmann.

Definizione 2.10. *Il processo di Markov si dice **all'equilibrio** se vale:*

$$\sum_{\nu} p_{\mu} P_{\mu\nu} = \sum_{\nu} p_{\nu} P_{\nu\mu} \quad (2.13)$$

cioè se il sistema, mediamente, entra nello stato μ tante volte quante ne esce, per unità di tempo.

Dalla (2.7) insieme alla (2.13), si ricava la **condizione di equilibrio del processo di Markov**:

$$p_{\mu} = \sum_{\nu} p_{\nu} P_{\nu\mu} \quad (2.14)$$

La condizione (2.14), in generale, non assicura la convergenza del processo di Markov alla distribuzione di Boltzmann.

Sia $w_{\mu}(t)$ la probabilità che nel processo, al tempo t , lo stato sia μ , e sia $\mathbf{w}$ il vettore che raccoglie tali probabilità ($\dim(\mathbf{w}) = \text{card}(\Gamma)$). Data la matrice stocastica $\mathbf{P}$, vale:

$$\mathbf{w}(t+1) = \mathbf{P} \cdot \mathbf{w}(t) \quad (2.15)$$

Se il processo converge ad un equilibrio semplice:

$$\mathbf{w}(\infty) = \mathbf{P} \cdot \mathbf{w}(\infty) \quad (2.16)$$

Richiedendo che valga la (2.14) (cioè che il processo di Markov sia all'equilibrio), assicuriamo che esista almeno una distribuzione di probabilità di equilibrio semplice p_{μ} , ma, a rigore, non possono essere esclusi altri tipi di convergenza (ad esempio, ad un ciclo limite: $\mathbf{w}(\infty) = \mathbf{P}^n \cdot \mathbf{w}(\infty)$). Per eliminare la possibilità di convergenza a cicli limite, si impone la seguente condizione (**bilancio dettagliato**):

$$p_{\mu} P_{\mu\nu} = p_{\nu} P_{\nu\mu}, \quad \forall \mu, \nu \quad (2.17)$$

L'equazione indica che il *rate* con cui il sistema va da μ in ν è uguale a quello con cui passa da ν a μ ; in questo modo si esclude la convergenza a cicli limite. Riscrivendo la

(2.14) in forma matriciale:

$$\mathbf{p} = \mathbf{P} \cdot \mathbf{p} \quad (2.18)$$

Quindi, si conclude che $\mathbf{p}$ è un **vettore stazionario di probabilità della matrice stocastica $\mathbf{P}$** . Di conseguenza, **la distribuzione di equilibrio del processo di Markov** è proprio $\mathbf{p}$: $\mathbf{w}(\infty) = \mathbf{p}$.

Si può dimostrare che il rispetto del principio del bilancio dettagliato è strettamente connesso alla simmetria per inversione temporale dei processi fisici. Se il sistema è invariante per inversione temporale (cioè $T : t \rightarrow -t$ è una simmetria), allora il bilancio dettagliato è rispettato. Dal principio del bilancio dettagliato (2.17), imponendo che la distribuzione di equilibrio sia quella di Boltzmanan, si ricava:

$$\frac{P_{\mu\nu}}{P_{\nu\mu}} = \frac{p_\nu}{p_\mu} = e^{-\beta(E_\nu - E_\mu)} \quad (2.19)$$

La scelta delle probabilità di transizione per il processo di Markov deve quindi rispettare le due condizioni:

$$\left\{ \begin{array}{l} \frac{P_{\mu\nu}}{P_{\nu\mu}} = \frac{p_\nu}{p_\mu} = e^{-\beta(E_\nu - E_\mu)} \\ \sum_{\nu} P_{\mu\nu} = 1 \end{array} \right. \quad (2.20)$$

$$\sum_{\nu} P_{\mu\nu} = 1 \quad (2.21)$$

2.6. Tassi di accettazione

Si osserva che qualsiasi valore $P_{\mu\nu}$ fra 0 e 1 rispetta il bilancio dettagliato. Inoltre, una modifica di $P_{\mu\nu}$, seguita da una appropriata modifica di $P_{\nu\mu}$, non preclude il rispetto della (2.19). A partire da queste considerazioni, si può far vedere rigorosamente che esistono infinite possibili scelte del set $\{P_{\mu\nu}\}$ in accordo con il sistema (2.20)-(2.21).

Scriviamo ora la probabilità di transizione come prodotto di due termini:

$$P_{\mu\nu} = g_{\mu\nu} A_{\mu\nu}, \quad (2.22)$$

dove:

- $g_{\mu\nu}$ è la **probabilità di selezione** di ν come nuovo stato a partire da μ .
- $A_{\mu\nu}$ è il **tasso di accettazione** dello stato proposto ν .

La (2.19) diventa allora:

$$\frac{P_{\mu\nu}}{P_{\nu\mu}} = \frac{g_{\mu\nu}A_{\mu\nu}}{g_{\nu\mu}A_{\nu\mu}} = e^{-\beta(E_\nu - E_\mu)} \quad (2.23)$$

La simulazione Monte Carlo si baserà quindi su un algoritmo generatore di nuovi stati a partire da un dato stato, e l'accettazione degli stati proposti avverrà con tasso $A_{\mu\nu}$. Scegliere tassi di accettazione $A_{\mu\nu} \in [0, 1]$ bassi implicherebbe che il sistema rimarrebbe quasi sempre nello stato di partenza, e l'esplorazione dello spazio degli stati sarebbe assai limitata. Al contrario, avere tassi di accettazione sempre vicini ad uno porterebbe il sistema a campionare un gran numero di stati in Γ , in una passeggiata casuale simile a un moto casuale a temperatura molto elevata. Per una coppia di stati (μ, ν) , considerando che il bilancio dettagliato pone condizioni sul solo rapporto fra i due tassi $A_{\mu\nu}$ e $A_{\nu\mu}$, è buona regola porre il maggiore dei due uguale ad uno, ed il minore tale da rispettare la condizione del bilancio dettagliato.

2.7. Algoritmo di Metropolis

Sviluppato da *Metropolis et al.* nel 1953, è l'algoritmo standardizzato per MCMC più comunemente usato e noto.

- Le probabilità di selezione $g_{\mu\nu}$ dello stato ν dallo stato μ sono scelte uguali $\forall \nu \in \Gamma$. Se $\lambda = \text{card}(\Gamma)$ (Γ spazio delle fasi accessibili dal sistema), si può porre allora $g_{\mu\nu} = \frac{1}{\lambda}, \forall \nu \in \Gamma$. In realtà, basterebbe che ogni stato μ abbia lo stesso numero di stati ν che può raggiungere in un solo passo. La (2.23) diventa allora:

$$\frac{P_{\mu\nu}}{P_{\nu\mu}} = \frac{A_{\mu\nu}}{A_{\nu\mu}} = e^{-\beta(E_\nu - E_\mu)} \quad (2.24)$$

- Data una coppia di stati (μ, ν) , di energie (E_μ, E_ν) , si pone, come detto precedentemente, a uno il tasso di accettazione maggiore, e si sceglie l'altro in modo da rispettare il bilancio dettagliato:

$$A_{\mu\nu} = \begin{cases} e^{-\beta(E_\nu - E_\mu)} & \text{se } E_\nu > E_\mu \\ 1 & \text{altrimenti} \end{cases}$$

Se lo stato selezionato ha energia minore, esso viene accettato con probabilità unitaria; se ha invece energia maggiore, la probabilità di accettazione è non nulla, e coincide con l'esponenziale alla Boltzmann della differenza delle energie.

OSSERVAZIONE: Dimenticandosi per un attimo del contesto fisico, e considerando l'algoritmo come uno strumento per un problema di minimizzazione, si nota che, per valori di β molto grandi, cioè per temperature tendenti a zero, l'algoritmo accetta quasi esclusivamente transizioni verso stati ad energia inferiore, e imita quindi il comportamento di algoritmi di tipo *discesa del gradiente*. Al contrario, per β piccoli e temperature alte, l'algoritmo accetta con alta probabilità ogni stato ν proposto, e tende a comportarsi quindi alla stregua di una *random search*.

Si può facilmente verificare che l'algoritmo di Metropolis soddisfa la condizione di bilancio dettagliato.

2.8. Dalla fisica statistica ai sistemi complessi

I metodi Monte Carlo sono finora stati analizzati come ausilio per il calcolo di valori di aspettazione di grandezze fisiche di sistemi fisici statistici. Questa è solo una delle loro innumerevoli applicazioni. In generale, una simulazione Monte Carlo è un tipo di algoritmo combinatorio che usa il campionamento statistico per risolvere problemi non affrontabili analiticamente o con i classici metodi numerici.

L'uso del *random sampling* per la risoluzione di problemi matematici risale al XVIII secolo, quando non esistevano calcolatori capaci di implementare algoritmi. In particolare, metodi basati sul campionamento statistico erano utilizzati per la risoluzione di integrali di funzioni complicate o di integrali multidimensionali, e per la determinazione di costanti (come nel caso del metodo de "l'ago di Buffon" per il calcolo del π). Le due guerre mondiali e l'avvento di calcolatori sempre più efficienti portarono alla fioritura dei metodi MC. Enrico Fermi, con i "ragazzi di via Panisperna", applicò per primo (senza pubblicarli) metodi MC per lo studio di fenomeni di diffusione neutronica, negli anni Trenta. Tali metodi furono reinterpretati ed utilizzati negli anni Quaranta da S. Ulam, N. Metropolis e J. von Neumann nello studio di armi termonucleari (prima della bomba atomica, poi della bomba H) ai laboratori nazionali di Los Alamos (New Mexico).

Da quel momento in poi, i campi di applicazione delle simulazioni MC sono progressivamente aumentati, arrivando a comprendere lo studio dell'intelligenza artificiale, dei prezzi delle azioni, delle previsioni di vendita, della gestione dei progetti; finanche ambiti come le scienze climatiche, la computer-grafica e la biologia computazionale sono stati arricchiti dai metodi MC.

Anche i *sistemi complessi* possono essere analizzati in questo modo. Un sistema complesso è un sistema con una *non separabilità* bidirezionale fra le identità delle parti e del sistema tutto. Quindi, non solo l'identità del tutto è determinata dai costituenti, ma anche l'identità delle parti è determinata dal tutto per via delle loro mutuali interazioni.

Esempi di sistemi complessi si trovano in economia, scienze sociali, climatologia, geologia e biologia, oltre che in fisica.

Un passo fondamentale nella modellizzazione di sistemi complessi è l'**ottimizzazione**. Per ottimizzazione si intende la ricerca di un minimo o di un massimo di una funzione continua di più variabili. Nell'ambito dell'*ottimizzazione combinatoria*, l'utilizzo di metodi MCMC risulta estremamente comune, come si avrà modo di vedere per il *Simulated Annealing* come algoritmo di approssimazione.

3 | *Traveling Salesman Problem* (TSP)

Il problema del commesso viaggiatore è il più noto problema di ottimizzazione combinatoria. Può essere formulato come segue:

Un commesso viaggiatore deve visitare n città. Qual è la sequenza di città che minimizza la distanza percorsa dal commesso?

Un qualsiasi problema di ottimizzazione combinatoria si definisce mediante una coppia (Γ, C) , dove Γ è lo spazio delle configurazioni possibili del sistema, e C è una **funzione costo**, equivalente all'hamiltoniana (energia) nel contesto della meccanica statistica. L'obiettivo dell'ottimizzazione è quello di trovare la configurazione che minimizza la funzione costo:

$$C_{opt} := \min_{j \in \Gamma} C(j) \quad (3.1)$$

Nel caso del *TSP*:

- Lo spazio delle configurazioni è costituito da vettori i cui elementi sono le città che, in ordine, sono visitate dal commesso viaggiatore.
- Il costo di un elemento di Γ coincide con la distanza geodetica totale corrispondente allo specifico percorso:

$$C = \sum_i d_i, \quad (3.2)$$

con d_i distanza fra due città della sequenza.

3.1. Cenni di Teoria della Complessità Computazionale

La Teoria della Complessità Computazionale si occupa dell'analisi delle minime risorse (tempo di calcolo e memoria) necessarie per l'esecuzione di un algoritmo costruito per la risoluzione di un problema.

Un concetto astratto fondamentale nella teoria della computazione è quello di **macchina di Turing** (*A. Turing, 1936-7*). Essa è immaginabile come una macchina automatica (*a-machine*) caratterizzata da un set di configurazioni di cardinalità finita $\{q_1, \dots, q_n\}$, alimentata da un nastro monodimensionale infinito, diviso in quadrati, ciascuno contenente un simbolo. In ogni momento la macchina scansiona il contenuto di un *un solo* quadrato r . La macchina è automatica, quindi il suo comportamento in un dato istante è *unicamente* determinato dalla configurazione e dal simbolo correnti (**condizione di determinazione**, *macchina di Turing deterministica*). Per **macchina di Turing non deterministica** si intende invece una macchina di Turing per la quale lo stato successivo non è completamente determinato dalla coppia (*stato corrente, simbolo corrente*). *Si può dimostrare che qualsiasi algoritmo di calcolo è implementabile da una macchina di Turing, nonostante la sua semplicità concettuale.*

I problemi di ottimizzazione sono usualmente suddivisi in **classi di complessità** sulla base delle risorse necessarie per la risoluzione del problema mediante l'algoritmo più prestante noto.

- **Problema P** (*Polynomial*). Può essere risolto in *tempo polinomiale* da una macchina di Turing deterministica: esiste un algoritmo risolutivo tale che il numero di *step* necessari all'algoritmo è limitato da una funzione polinomiale in n , dove n è la lunghezza dell'input del problema.
- **Problema NP** (*Non-deterministic Polynomial*). La soluzione può essere ottenuta in *tempo polinomiale* mediante una macchina di Turing non deterministica.
- **Problema NP-difficile**. L'algoritmo di risoluzione può essere modificato per risolvere qualsiasi problema NP o P.

Un problema che è NP ed NP-difficile è detto **NP-completo**. Risulta elementare dimostrare che vale la seguente relazione fra classi:

$$P \subseteq NP$$

La validità della relazione opposta: $NP \subseteq P$ è tuttora ignota. Non è infatti noto se algoritmi di tempo polinomiale saranno mai trovati per problemi NP-completi; determinare che tali problemi sono trattabili in questo modo porterebbe ad asserire che $P \equiv NP \equiv NP - \text{completo}$. Il problema "*P vs NP*" rappresenta uno dei sette problemi per il millennio, importanti problemi matematici irrisolti elencati dall'istituto matematico Clay di Cambridge.

Il TSP è un problema NP-completo.

3.2. Algoritmo di Metropolis per il TSP

Si consideri ora l'algoritmo di Metropolis precedentemente introdotto nell'ambito dei Metodi Monte Carlo a catena di Markov. Si vuole utilizzare l'algoritmo per trovare il minimo della funzione costo nel *Traveling Salesman Problem*. Come già spiegato nel paragrafo 2.7, l'algoritmo di Metropolis consiste, nell'ambito dei metodi MCMC applicati alla meccanica statistica, nella scelta delle probabilità di transizione $P_{\mu\nu} = g_{\mu\nu}A_{\mu\nu}$:

- $g_{\mu\nu}$, probabilità di selezione dello stato, è uguale per ogni stato ν ;
- $A_{\mu\nu}$, tasso di accettazione, vale 1 se $E_\nu < E_\mu$, $e^{-\beta(E_\nu - E_\mu)}$ altrimenti.

Quali sono le corrispondenze che sussistono fra le simulazioni Monte Carlo per sistemi fisici "statistici" (composti cioè da un elevato numero di componenti) e quelle per problemi di ottimizzazione?

1. Le configurazioni del problema di ottimizzazione (ad esempio i diversi itinerari nel caso del TSP) sono analoghe ai microstati del sistema fisico.
2. La funzione costo di una certa configurazione (nel caso del TSP è la distanza totale dell'itinerario) corrisponde all'energia di uno stato del sistema fisico.
3. Il parametro di controllo c nella simulazione MC di un'ottimizzazione corrisponde alla temperatura termodinamicamente intesa del sistema fisico.

Nota: le simulazioni che seguono e le rispettive righe di codice sono state elaborate con il software "Wolfram Mathematica", che fa uso del linguaggio "Wolfram Language".

3.2.1. Algoritmo di Metropolis per il TSP: comportamento al variare del parametro di controllo

L'obiettivo di questa prima sezione è quello di costruire l'algoritmo MC, eseguirlo, ed analizzare i risultati della simulazione MC (che avviene **a β fissato**) *al variare del parametro β* , definito come $\beta = \frac{1}{k_B c}$, con c parametro di controllo.

N.B.: In questa sezione e nella successiva considereremo una versione particolare del TSP, in cui vincolo sia quello di mantenere costante la città di partenza.

I diversi valori di beta con cui effettueremo la simulazione sono memorizzati nel vettore:

```
vettBeta = {0.0001, 0.001, 0.01, 0.1};
```

Un generico elemento dello spazio delle configurazioni (vettore di stato) è su Mathematica una lista ordinata i cui elementi sono liste, ciascuna corrispondente ad una città dell'i-

tinero e contenente il suo nome, latitudine e longitudine. Sono inoltre inizializzate le liste seguenti, che, **per ogni valore di β** :

- *minimi*: memorizza i percorsi ottimali;

N.B.: Nella presente trattazione, per "percorso ottimale" si intende l'itinerario che, fra quelli esplorati durante le simulazioni, ha la distanza minima; quest'ultima verrà chiamata "distanza ottimale".

- *storia*: presenta la successione delle distanze accettate;
- *numAccettazioni*: contiene il numero di accettazioni (servirà per calcolare la percentuale di accettazione);
- *varCostoMedio*: contiene la somma delle differenze di distanza per gli scambi di percorso accettati (servirà per calcolare la variazione media di costo).

Si stabilisce poi il valore n di città da includere nel problema. Ad esempio:

`n = 10;`

L'algoritmo costruito realizza una catena di Markov omogenea mediante una funzione di creazione di un nuovo stato random a partire dallo stato presente. Illustriamo come opera tale funzione, chiamata:

`creaNuovoStato[stato_]`

A partire dallo stato presente, la funzione mantiene costante la prima città del percorso e cambia casualmente l'ordine delle altre città dell'itinerario. Tale operazione avviene mediante generazione di due numeri casuali grazie al comando:

`{rand1, rand2} = RandomInteger[{2, n}, 2];`

In particolare, la funzione inverte il tratto di percorso fra le città in posizione *rand1* e *rand2*, e lascia invariati il tratto dalla prima città a quella in posizione *rand1-1* e quello dalla città in posizione *rand2+1* all'ultima città.

Inoltre, l'algoritmo implementa la regola di accettazione di Metropolis: definita la variabile *varCosto* che memorizza volta per volta la differenza di distanza fra i due itinerari, l'accettazione o il rifiuto della nuova configurazione avviene mediante la verifica di una delle due condizioni seguenti:

`If[varCosto < 0 || Exp[-vettBeta[[j]]*varCosto] > RandomReal[],`

Quindi, l'accettazione dello stato proposto è certa se esso ha distanza totale minore, altrimenti avviene con probabilità data dalla regola di Metropolis. Fissato per ogni β lo

stesso stato iniziale, l'operazione viene ripetuta un numero di volte *rip*, scelto in modo da garantire che a fine simulazione l'equilibrio venga raggiunto con buona probabilità. Senza approfondire sui criteri che indicano come scegliere il parametro *rip* per arrivare all'equilibrio, si pone:

$$rip = 10 * n; (*numero di ripetizioni per ciascun valore di beta*)$$

Per una trattazione più precisa di questo aspetto, si dovrebbe stabilire il valore della grandezza **Monte Carlo steps per site** (MCSS); essa equivale al numero minimo di tentativi da eseguire perchè tutte le variabili di stato del sistema siano aggiornate. Spesso il MCSS coincide con il numero di elementi nel vettore di stato. In questo caso, appare quindi ragionevole supporre:

$$1MCSS = n, \quad (3.3)$$

e porre

$$rip = 10MCSS, \quad (3.4)$$

per avere una buona probabilità di convergenza dell'algoritmo all'equilibrio a fine simulazione. Analizziamo adesso i risultati ottenuti dalle simulazioni in un caso specifico di esecuzione del codice.

```
Vettore di stato iniziale: Zurich, Munich, Amsterdam, Sofia, Lisbon,
Berlin, Porto, Reykjavik, Zagreb, Edinburgh
Percorso ottimale: Zurich, Munich, Zagreb, Sofia, Berlin, Amsterdam,
Lisbon, Porto, Edinburgh, Reykjavik
Percentuali di accettazione, per ogni beta: {92.0,60.0,34.0,16.0}
Distanze ottimali trovate, per ogni beta (km): {13012.,9291.18,8449.75,
9111.79}
Variazione media di distanza per le scelte accettate, per ogni beta:
{-30.3916,-77.149,-329.906,-392.639}
```

Il comportamento che mostrano gli esiti nelle simulazioni è in linea con le aspettative. Consideriamo la figura 3.1. In generale, per β bassi, cioè parametri di controllo alti, il sistema accetta con elevata probabilità anche proposte peggiorative in termini di distanza; al contrario, con β alti, diminuisce drasticamente la probabilità di accettare configurazioni peggiorative proposte dall'algoritmo di generazione, e la percentuale di accettazione scende. Nella simulazione, questo comportamento è rispettato, con percentuale che scende dal 92% per $\beta = 0.0001$ al 34% per $\beta = 0.1$.

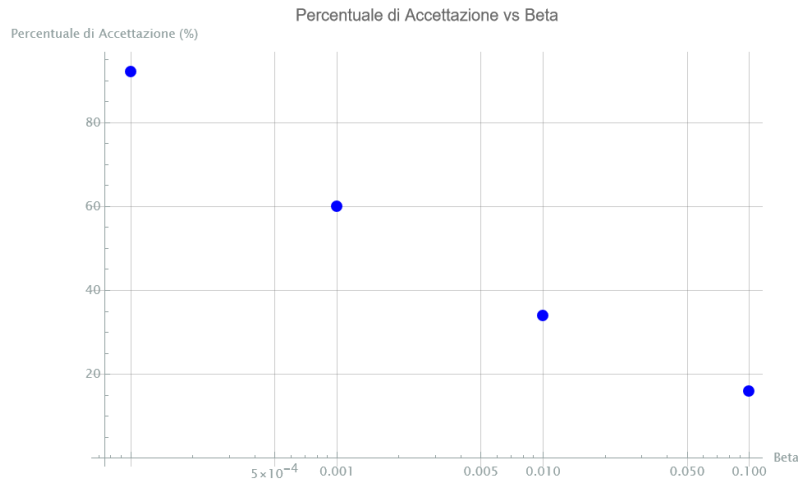


Figura 3.1: Su scala semilogaritmica è riportata per i quattro valori di β la percentuale di accettazione di nuovi stati proposti dall'algoritmo di generazione.

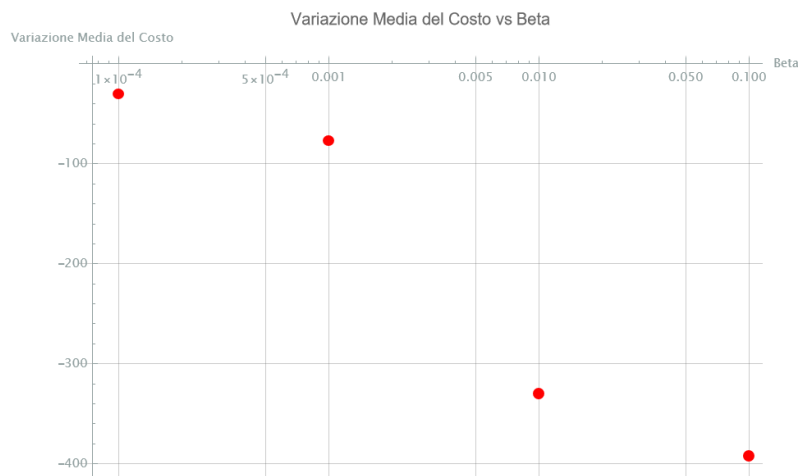


Figura 3.2: Su scala semilogaritmica è riportata per i quattro valori di β la variazione media del costo per le accettazioni.

Anche nella figura 3.2 il trend è quello atteso. Per β bassi, ci aspettiamo che la variazione media di costo sia molto piccola in modulo rispetto all'ordine di grandezza della distanza totale del percorso; questo avviene perchè l'elevata probabilità di accettazione di proposte peggiorative determina la presenza di tante variazioni di distanza positive, oltre a quelle negative accettate con probabilità unitaria. Per β alti la pressochè nulla probabilità di accettazione di proposte peggiorative (cioè con variazione positiva della distanza) implica che ogni accettazione ha variazione di distanza negativa; ciò porta la variazione media a salire in modulo. Nella simulazione, l'andamento è (come previsto) decrescente, a partire

da -30 km per $\beta = 0.0001$ a -393 km per $\beta = 0.1$.

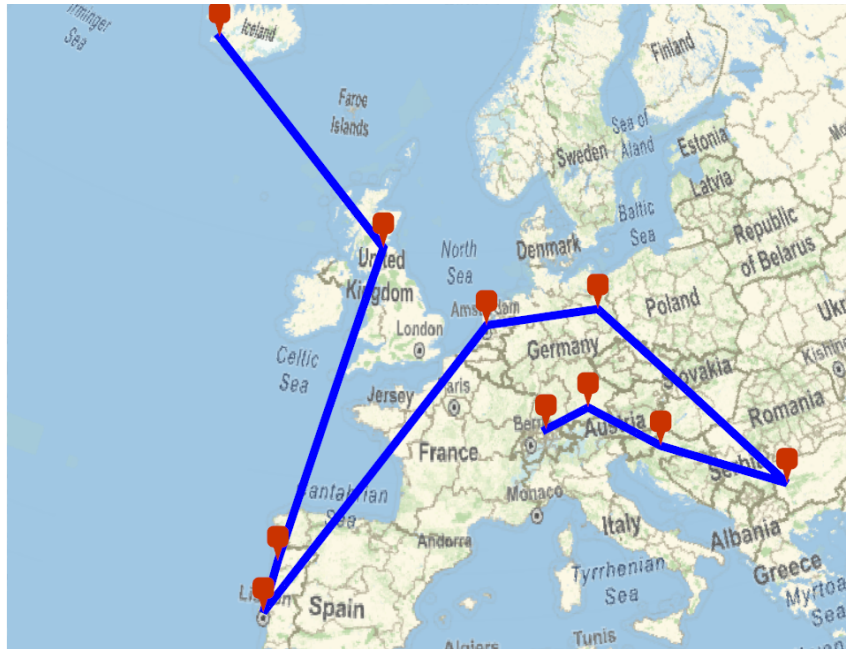


Figura 3.3: Rappresentazione geografica del percorso ottimale trovato

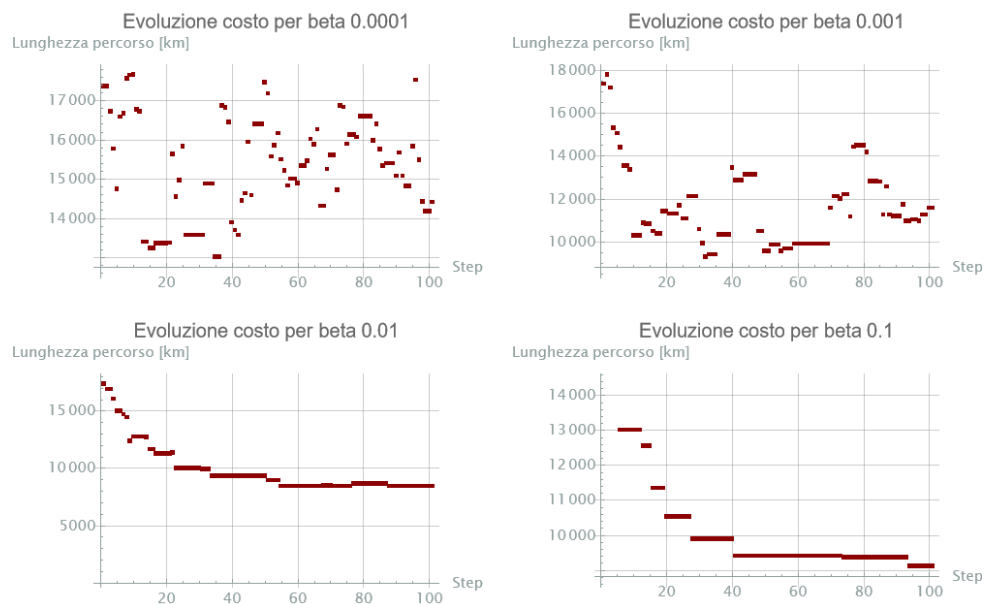


Figura 3.4: Evoluzioni, per ogni β , delle distanze delle configurazioni accettate, con $n=5$.

Analizziamo ora i quattro grafici in figura 3.4. Sull'asse delle ordinate troviamo le lunghezze dei percorsi accettati, ovvero i costi degli stati che la simulazione ha esplorato nello spazio delle configurazioni.

- $\beta = 0.0001$: a fine simulazione non è presente una convergenza verso un ottimo; l'esplorazione dello spazio delle configurazioni è guidata infatti da un criterio di accettazione delle proposte che dà praticamente sempre esito positivo.
- $\beta = 0.001$: la simulazione mostra un trend discendente all'aumentare delle iterazioni, senza tuttavia presentare una convergenza qualitativamente ben distinguibile;
- $\beta = 0.01$: la simulazione converge visibilmente verso un ottimo;
- $\beta = 0.1$: analoga considerazione del punto precedente, ma l'ottimo cui l'algoritmo converge è diverso.

Evidentemente, all'aumentare del parametro di controllo, aumenta il numero di scelte peggiorative, e l'algoritmo necessita quindi di più passi per convergere ad un equilibrio. Quello che si nota è che i percorsi ottimali trovati per ogni β restituiti dall'output:

Distanze ottimali trovate, per ogni beta (km): {13012.0,9291.18,8449.75,9111.79}

sono tutti diversi. Come ci si spiega ciò? Per rispondere a questa domanda effettueremo altre due simulazioni.

- Avviamo un'altra simulazione con $n=5$. Si trova:

Vettore di stato iniziale: Dublin, Moscow, Cologne, Vilnius, Lyon

Percorso ottimale: Dublin, Lyon, Cologne, Vilnius, Moscow

Percentuali di accettazione, per ogni beta: {98.0,70.0,32.0,26.0}

Distanze ottimali trovate, per ogni beta(km): {4086.04,3850.8,3850.8,3850.8}

Variazione media di distanza per le scelte accettate, per ogni beta(km): {-33.7324,-43.8992,-120.54,-113.843}

Nel caso a n basso, gli ottimi trovati per i tre valori massimi di β hanno tutti lo stesso valore. La distanza ottimale a cui perviene la simulazione a β minimo (cioè parametro di controllo massimo) è maggiore di quella trovata nelle tre simulazioni successive. Come si vede dalla figura 3.5, la prima simulazione non raggiunge l'equilibrio dopo 10 *MCSS* ripetizioni: il valore ottenuto di 4084.06 *km* non è probabilmente un minimo, nè locale nè globale. **Nel caso di poche città, il sistema possiede quindi ragionevolmente un solo minimo, che è quindi globale.**

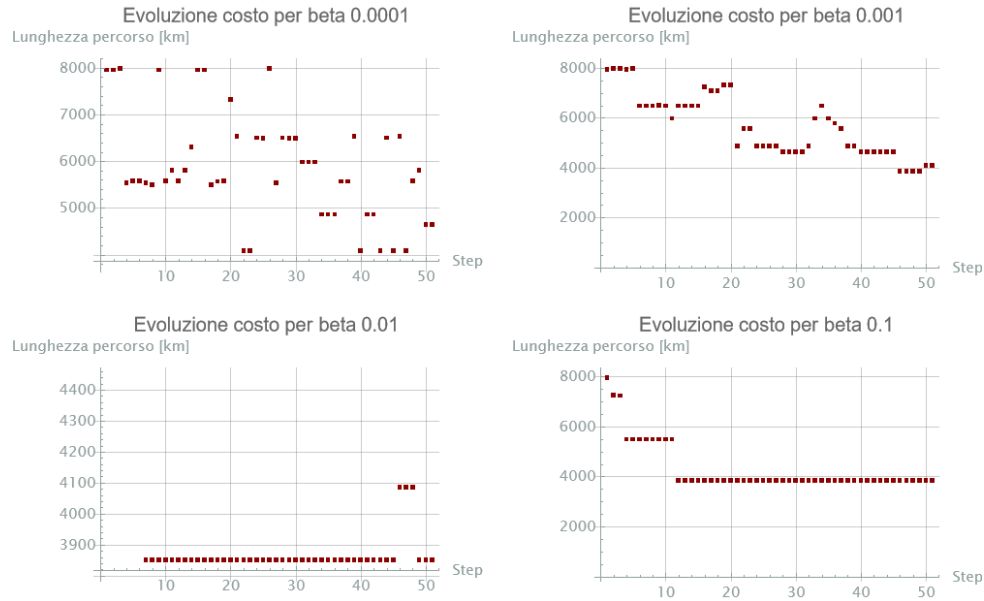


Figura 3.5: Evoluzioni, per ogni β , delle distanze delle configurazioni accettate, con $n=5$.

- Effettuiamo una simulazione con $n=25$.

Vettore di stato iniziale: Edinburgh, Ljubljana, Sarajevo, Lisbon, Moscow, Bratislava, Kiev, Monaco, Helsinki, Budapest, Vilnius, Cologne, Porto, Copenhagen, Vienna, Luxembourg, Amsterdam, Madrid, Reykjavik, Warsaw, Seville, Bern, Berlin, Stockholm, Oslo

Percorso ottimale: Edinburgh, Reykjavik, Amsterdam, Cologne, Luxembourg, Bern, Monaco, Ljubljana, Vienna, Budapest, Bratislava, Berlin, Warsaw, Vilnius, Copenhagen, Oslo, Stockholm, Helsinki, Moscow, Kiev, Sarajevo, Madrid, Lisbon, Porto, Seville

Percentuali di accettazione, per ogni beta(km): {98.4, 62.4, 23.2, 17.6}

Distanze ottimali trovate, per ogni beta(km): {27354.7, 22670., 16233.3, 15228.5}

Variazione media di distanza per le scelte accettate, per ogni beta(km): {-109.36, -223.083, -859.825, -1414.04}

Con n grande, si nota invece che ogni ripetizione dell'algoritmo con un diverso valore di β restituisce un valore diverso. Mentre in 10MCSS le prime due simulazioni non hanno raggiunto un equilibrio (vedi figura 3.6), le ultime simulazioni hanno raggiunto due valori di equilibrio, ma diversi. **Evidentemente, all'aumentare del numero di città nel percorso, aumenta il numero di minimi locali.** Quest'ultima osservazione emerge analogamente nella sezione 3.2.2.

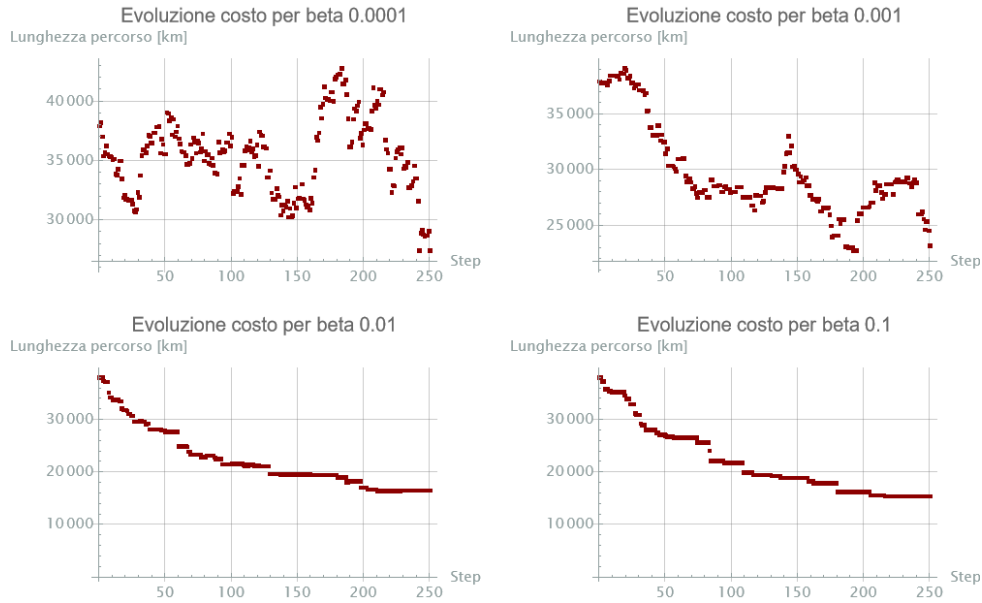


Figura 3.6: Evoluzioni, per ogni β , delle distanze delle configurazioni accettate, con $n=25$.

3.2.2. Algoritmo di Metropolis per il TSP: comportamento al variare dello stato iniziale

Individuiamo adesso il valore del parametro di simulazione $\beta=0.1$ che sia "alto", cioè che, nella sezione 3.2.1, abbia mostrato un'evoluzione della funzione costo strettamente decrescente (vedi figura 3.5). Ciò corrisponde ad un basso parametro di controllo, che implica una bassa probabilità di accettare scelte peggiorative ed un andamento dell'algoritmo tendente a quello di una semplice discesa del gradiente. Scelte n città, effettuiamo con tale β un numero $nTest$ di simulazioni, ciascuna che parte da uno stato diverso, ma con Roma che rimane sempre la prima città del percorso fra le n città. Per ogni simulazione il parametro rip (scelto come nella sezione precedente) indica il numero di passi effettuati. Poniamo $nTest = 6$.

- Con $n=5$ si ottiene:

Vettore di stato iniziale: Rome, Vienna, Warsaw, Zurich, Frankfurt
 Percorso ottimale trovato: Rome, Zurich, Frankfurt, Vienna, Warsaw

Con un numero così ridotto di città, la funzione distanza totale ha un solo minimo di 2143 km, che viene identificato in ognuna delle simulazioni eseguite.

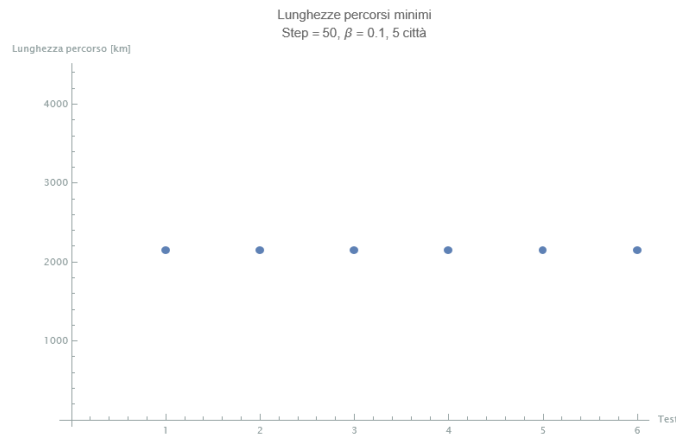


Figura 3.7: Lunghezze dei percorsi ottimali trovati in ogni test, con $n=5$.

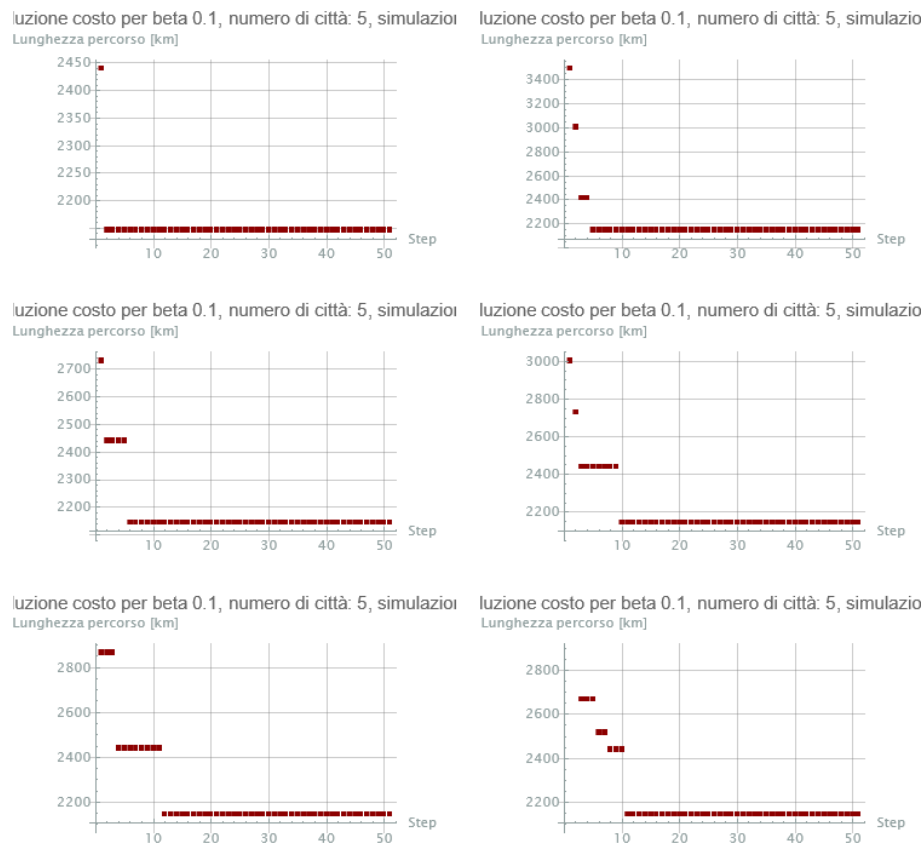


Figura 3.8: Evoluzione della distanza totale in ogni simulazione, con $n=5$.

- Con $n=10$ si ottiene:

Vettore di stato iniziale: Rome, Ljubljana, Gothenburg, Barcelona, Vilnius, Edinburgh, Moscow, Tallinn, Marseille, Dublin
 Percorso ottimale trovato: Rome, Barcelona, Marseille, Ljubljana,

Vilnius, Moscow, Tallinn, Gothenburg, Edinburgh, Dublin

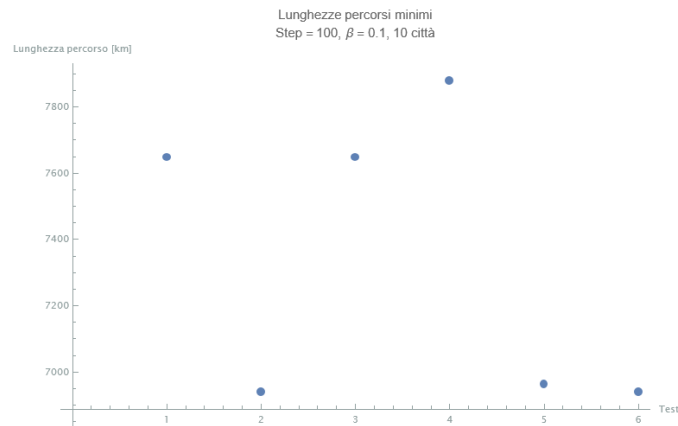


Figura 3.9: Lunghezze dei percorsi ottimali trovati in ogni test, con $n=10$.

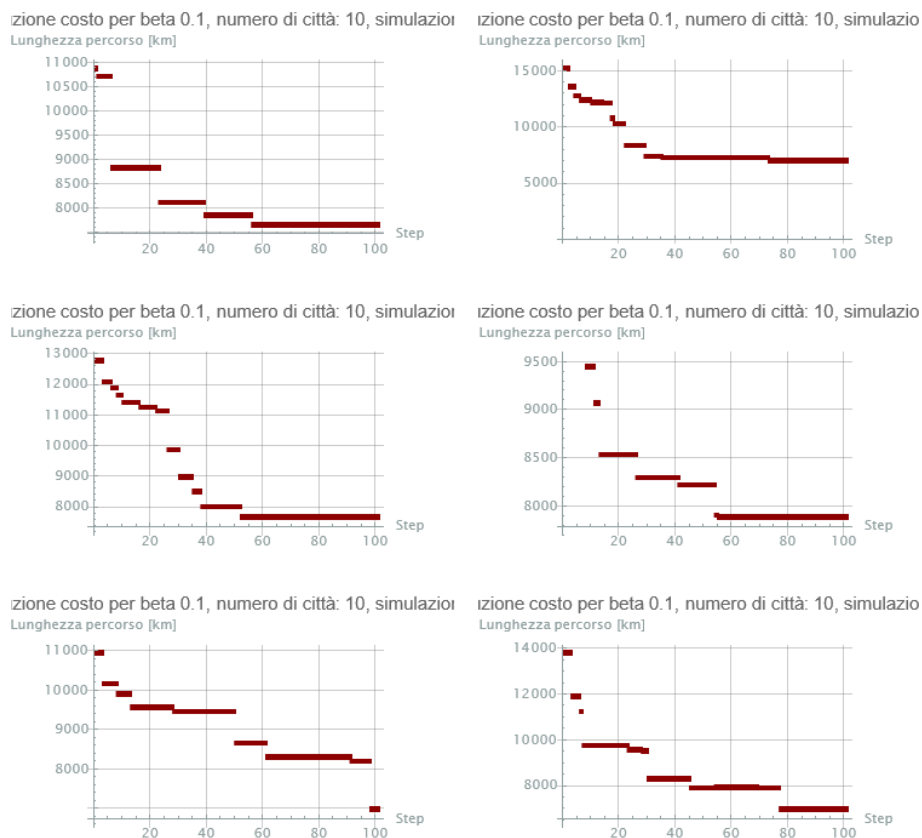


Figura 3.10: Evoluzioni dei percorsi ottimali in ogni simulazione, con $n=10$.

Si nota che, per numeri maggiori di n , gli ottimi trovati in ogni test sono diversi. Nel caso in questione due dei sei test trovano la migliore approssimazione del minimo assoluto della funzione distanza, cioè 6940 *km*.

La conclusione che si trae è la seguente: per valori "piccoli" del numero di città, per i quali $C = \sum_i d_i$ ha un unico minimo, l'algoritmo di Metropolis iterato per un numero adeguato di ripetizioni (da far convergere all'equilibrio) porta efficacemente al minimo della funzione da ottimizzare. Al contrario, al crescere di n , la comparsa di minimi relativi rende tale algoritmo inaffidabile come strumento di ottimizzazione. Partendo da stati iniziali diversi si trovano infatti ottimi diversi, e non si ha la certezza che il minimo dei valori trovati sia quello assoluto per C .

Per risolvere il problema, è necessario utilizzare algoritmi di ottimizzazione più sofisticati. Nell'ambito degli algoritmi di approssimazione per l'ottimizzazione combinatoria, il *Simulated Annealing* è l'algoritmo a cui faremo riferimento nella sezione 3.3.

3.2.3. Algoritmo di Metropolis per il TSP: variante a città iniziale variabile

Analizziamo adesso una variante dell'algoritmo finora implementato nelle sezioni 3.2.1 e 3.2.2. Eliminiamo il vincolo per cui la prima città fra quelle selezionate nel percorso casuale iniziale rimane fissata, e lasciamo che il punto di partenza dell'itinerario sia variabile durante l'ottimizzazione. In particolare, eseguiamo la simulazione con un numero elevato di città (tale da comportare la presenza di minimi locali):

$$n = 20$$

Effettuiamo simulazioni a singolo valore di beta. Scegliamo un valore di beta "alto", cioè un valore che, da quanto emerge nella sezione 3.2.1, presenta una distinta convergenza a un ottimo dopo il numero di step scelto:

$$rip = 10 \cdot n.$$

Un valore che soddisfa la precedente richiesta è 0.1:

$$\beta = 0.1.$$

Eseguendo la simulazione, la convergenza all'ottimo emerge dal grafico 3.11.

L'output restituisce:

```
Vettore di stato iniziale: Reykjavik, Lyon, Edinburgh, Ljubljana, Milan,
Oslo, Paris, Bucharest, Athens, Dublin, Monaco, Helsinki, Copenhagen,
Prague, Vienna, Madrid, Bratislava, Seville, London, Marseille
Distanza iniziale (km): 28341
Percorso ottimale: Seville, Madrid, Marseille, Monaco, Lyon, Paris,
London, Copenhagen, Prague, Helsinki, Oslo, Edinburgh, Reykjavik,
```

Dublin, Milan, Vienna, Ljubljana, Bratislava, Bucharest, Athens

Distanza ottima (km): 14092

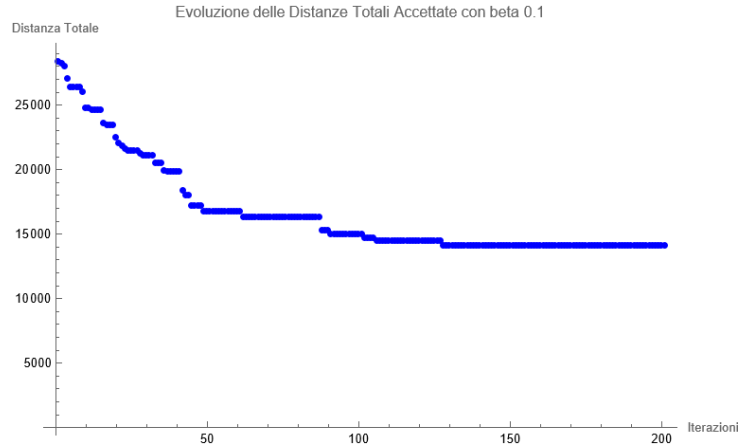


Figura 3.11: Evoluzione delle distanze accettate (km) dall'algoritmo durante i 200 step, con $\beta = 0.1$.

Focalizziamoci adesso sul seguente aspetto. Nel susseguirsi delle iterazioni, come si comportano i punti di partenza e di arrivo del percorso fra le 20 città? Per analizzare questo punto, si implementa nel codice la costruzione di due istogrammi.

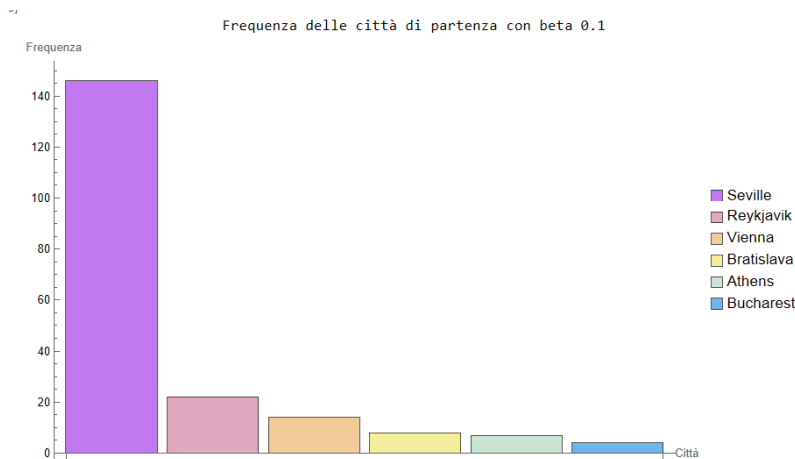


Figura 3.12: Istogramma delle città di partenza.

L'istogramma 3.12 mostra le frequenze assolute di comparsa delle città come punti di partenza dell'itinerario. Si nota che, fra le 20 città del percorso, solo 6 presentano frequenze assolute non nulle. Inoltre, fra queste, Siviglia è largamente dominante sulle altre, con una frequenza assoluta di 146, e una conseguente frequenza relativa di $\frac{146}{200} = 0.73$.

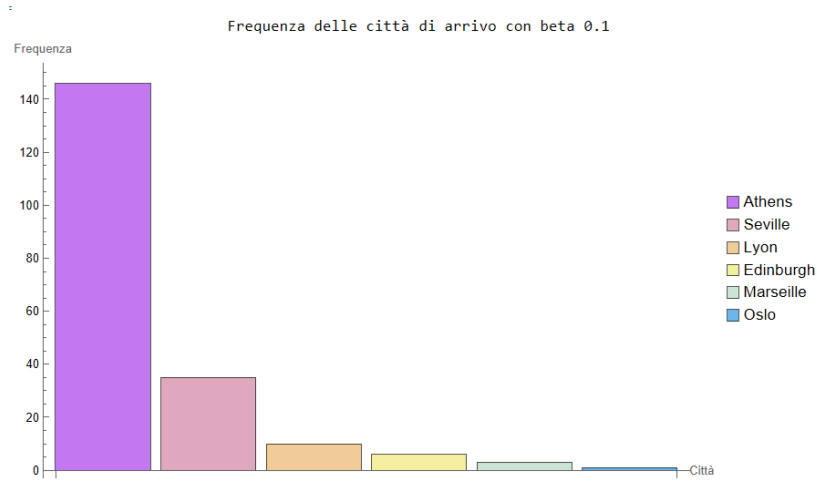


Figura 3.13: Istogramma delle città di arrivo.

L'istogramma 3.13 mostra invece le frequenze assolute relative ai punti di arrivo. Anche qui solo 6 delle 20 città hanno frequenze non nulle, e Atene domina con frequenze assolute e relative rispettivamente di 146 e 0.73. Qualitativamente, risulta evidente che non è presente molta variabilità sui punti iniziale e finale.

Per quantificare meglio questa considerazione, utilizziamo un indice noto come **coefficiente di Gini** (o indice di Gini); quest'ultimo è spesso utilizzato in economia, per descrivere aspetti come le diversità di reddito e ricchezza in una nazione. Tuttavia, esso può essere utilizzato, più in generale, **quando si vuole studiare la dispersione di una distribuzione con classi di dati categorici in statistica descrittiva**.

Il coefficiente di Gini può assumere valori fra 0 e 1:

- $G = 0$ indicherebbe una perfetta omogeneità nella distribuzione (cioè, tutte le città compaiono come partenza o arrivo lo stesso numero di volte);
- $G = 1$ mostrerebbe una totale inomogeneità della distribuzione (la stessa città compare tutte le volte come partenza o arrivo).

Senza entrare troppo nei dettagli teorici del coefficiente di Gini, utilizzeremo per il suo computo l'espressione riportata in [11].

Proposizione 3.1. *Siano $f_1, f_2, \dots$ le frequenze assolute (delle città di partenza o arrivo), $\bar{f}$ la media aritmetica di tali frequenze assolute, n il numero totale di città. L'indice di Gini G vale:*

$$G = \frac{1}{2n^2\bar{f}} \sum_{j=1}^n \sum_{i=1}^n |f_i - f_j| \quad (3.5)$$

Implementando nel codice il calcolo dell'indice di Gini, si ottiene:

Indice di Gini per le città di partenza: 0.631012

Indice di Gini per le città di arrivo: 0.68408

Si nota che i due indici sono entrambi superiori a 0.5. Ciò indica quindi due distribuzioni altamente diseguali, con una classe statistica (in questo caso una città) dominante.

La scarsa variabilità potrebbe essere dovuta a più fattori, fra cui i seguenti.

- Valore di β : il valore scelto per β è il più alto fra quelli usati nelle sezioni precedenti, e il parametro di controllo è quindi il più piccolo. L'esplorazione dello spazio delle fasi guidata dalla regola di Metropolis potrebbe dunque essere limitata da ciò.
- Spazio delle configurazioni: se lo stato iniziale si trova vicino ad un minimo, oppure se è presente un percorso minimo (o comunque ottimale) particolarmente dominante, la variabilità su punti di partenza e di arrivo sarebbe limitata. Se tale minimo fosse assoluto, la scarsa variabilità fra le città di partenza e di arrivo potrebbe indicare che l'algoritmo di Metropolis, anche con un numero elevato di città, riesca, dopo un numero appropriato di iterazioni (in modo da convergere ad un ottimo), quantomeno ad individuare i possibili punti di partenza e di arrivo che porterebbero al minimo assoluto. Quindi, nonostante con così tante città l'algoritmo di Metropolis sia incapace di trovare una buona approssimazione del minimo, esso riuscirebbe comunque ad individuare le due città che devono essere agli estremi del percorso per poter sperare di minimizzare la distanza. In realtà, nonostante questa eventualità possa verificarsi, non si può escludere a priori che le città di partenza e di arrivo individuate dalla simulazione MC siano corrispondenti ad un percorso ottimo la cui distanza ottimale sia un minimo locale, da cui la simulazione non si allontana (magari per via dell'alto β , a causa dello stato iniziale o per via dell'algoritmo che propone trasformazioni all'itinerario).

3.3. *Simulated Annealing per il TSP*

Il *Simulated Annealing* (che chiameremo per brevità SA) è un algoritmo di ottimizzazione combinatoria, ispirato al fenomeno dell'*annealing* dei solidi (vedi [2] per una trattazione teorica accurata). Applichiamo ora l'algoritmo al TSP.

Il codice scritto ed eseguito in *Mathematica* si basa su **tre cicli *for* annidati**, che ripercorrono i passi logici cruciali della simulazione. Essi, in ordine di comparsa (dal più "esterno" al più "interno"), sono:

1. For[itot = 1, itot <= ntot, itot++,

Il contatore *itot* fa riferimento al numero di volte *ntot* che l'algoritmo SA viene ripetuto, *partendo ogni volta da un diverso stato iniziale*.

2. For[*itemp* = 1, *itemp* <= *ntemp*, *itemp*++,

Il contatore *itemp* indica che il parametro β viene progressivamente aumentato (un numero *ntemp* di volte), con conseguente progressiva diminuzione del parametro di controllo, che chiameremo "temperatura" in analogia al caso fisico.

3. For[*isteps* = 1, *isteps* <= *nsteps*, *isteps*++,

Il contatore *isteps* si riferisce invece al numero di iterazioni in termini di MCSS , cioè al numero di scelte proposte per ogni valore di β .

Per quantificare la complessità computazionale affidata al calcolatore, si denomina il parametro

$$\chi := ntot \cdot ntemp \cdot nsteps,$$

che è uguale al numero di iterazioni totali della simulazione.

3.3.1. Schemi di raffreddamento della simulazione e andamento per n piccoli

Come noto, il SA è assimilabile ad una successione di algoritmi di Metropolis effettuati a temperature decrescenti. Emerge quindi la necessità di definire una modalità per l'innalzamento di β nel corso delle iterazioni del ciclo *For* corrispondente. Parlando dei possibili *schemi di raffreddamento* in relazione alla convergenza dell'algoritmo, si distingue fra:

- raffreddamento logaritmico:

$$\beta_n = \beta_0 \cdot \text{Log}(1 + itemp_n); \quad (3.6)$$

- raffreddamento geometrico:

$$\beta_n = \beta_0 \cdot itemp_n. \quad (3.7)$$

Effettuiamo adesso le simulazioni, ponendo: $\chi=1000$. Scegliamo infatti:

$$ntot = 2, \quad ntemp = 10, \quad nsteps = 10 \cdot 5 = 50 \quad (3.8)$$

Nell'output comparirà ogni accettazione secondo il comando seguente:

```
Print[{counter, itot, itemp, isteps, "Distanza ottimale:",
migliorDistanza}];
```

Partiamo anzitutto con un numero basso di città: $n=5$. Ricordiamo che in questi range di n anche il semplice algoritmo Monte Carlo trovava adeguatamente il minimo, che è unico. Ci aspettiamo quindi che il SA porti a risultati analoghi a quelli del MC, e che ci sia una blanda dipendenza dell'esito della simulazione dal tipo di raffreddamento adottato. Con un raffreddamento geometrico, ad esempio, si ottiene l'output:

```
Percorso iniziale: Rome, Belgrade, Athens, Barcelona, Berlin
Distanza iniziale(km): 4902.42
{1,1,1,11,Distanza ottimale(km):,4172.97}
Percorso ottimale: Rome, Barcelona, Berlin, Belgrade, Athens
Distanza ottimale(km): 4172.97
```

Come emerge dalla simulazione, il minimo è unico. La reale utilità di questo algoritmo può essere sfruttata solo incrementando il numero di città.

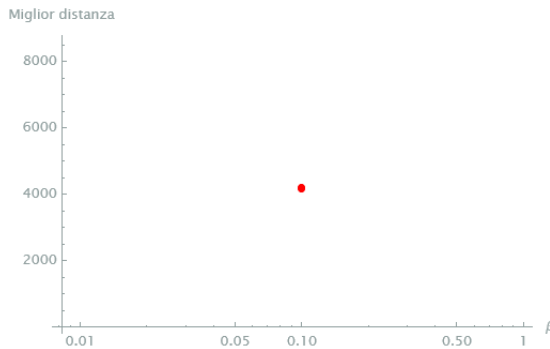


Figura 3.14: Distanze ottimali accettate al variare dei β utilizzati, su scala semi-logaritmica.

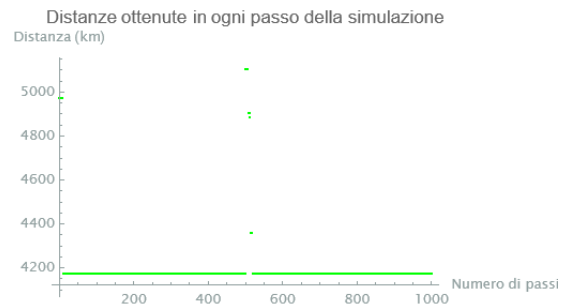


Figura 3.15: Distanze ottenute nei χ passi della simulazione, con raffreddamento geometrico e $n=5$.

3.3.2. Simulazioni SA con n elevato e schema di raffreddamento geometrico

Poniamo adesso:

$$n = 20, \text{ } n_{tot} = 2, \text{ } n_{temp} = 10, \text{ } n_{steps} = 10 \cdot 20 = 200$$

Da cui: $\chi = 4000$. Il raffreddamento è geometrico, del tipo:

$$\beta_n = \beta_0 \cdot itemp_n, \quad (3.9)$$

e viene scelto $\beta_0 = 0.1$.

Percorso iniziale: Rome, Geneva, Kiev, Copenhagen, Athens, Helsinki,
Bratislava, Berlin, Lisbon, Istanbul, Edinburgh, Barcelona, Belgrade,
Bern, Budapest, Ljubljana, Bucharest, Dublin, Frankfurt, Brussels
Distanza iniziale(km): 29277.2

Abbiamo omesso per brevità i numerosi output relativi alle singole proposte accettate dall'algoritmo.

Percorso ottimale: Rome, Bern, Geneva, Barcelona, Lisbon, Dublin,
Edinburgh, Brussels, Frankfurt, Copenhagen, Berlin, Ljubljana,
Bratislava, Budapest, Belgrade, Athens, Istanbul, Bucharest, Kiev,
Helsinki
Distanza ottimale(km): 11757.2

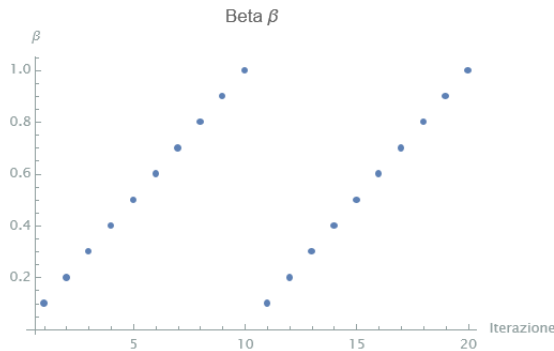


Figura 3.16: Andamento di β nel susseguirsi delle iterazioni.

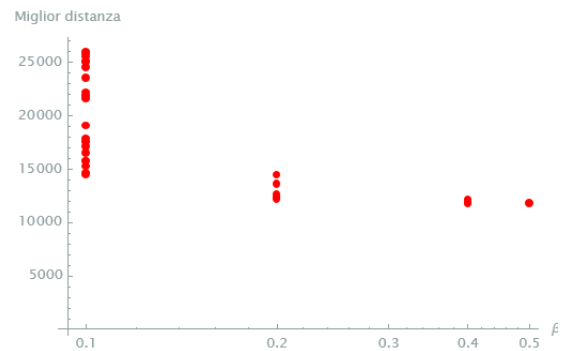


Figura 3.17: Distanze ottimali accettate al variare dei β utilizzati.

Risulta chiaro dalla figura 3.18 che, fra le due iterazioni a partire da due stati iniziali diversi, solo la seconda porta al valore ottimo trovato nella presente simulazione. Poichè l'algoritmo SA è un algoritmo di approssimazione, non sarebbe rigoroso affermare che esso sia il minimo assoluto del costo, ma solo una sua approssimazione, la migliore di cui disponiamo a fine simulazione.

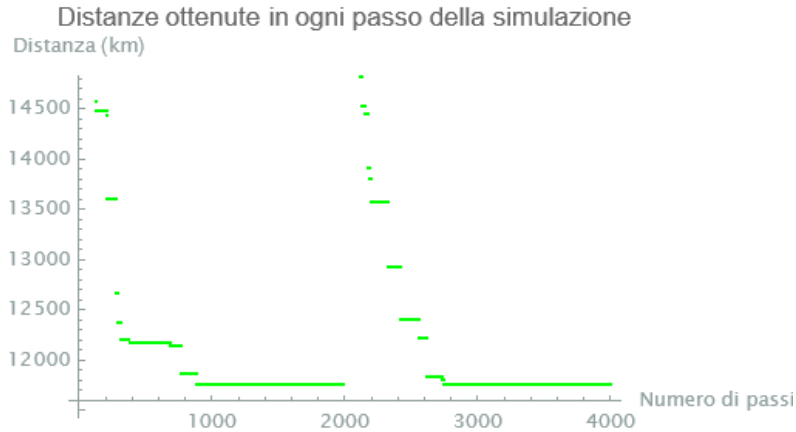


Figura 3.18: Distanze ottenute nei singoli passi della simulazione.

Proviamo ora ad eseguire più volte il codice appena presentato (doppia ripetizione dell'algoritmo SA a partire da due stati iniziali diversi), partendo ogni volta da un percorso iniziale casuale diverso (ma contenente sempre le stesse città). Le città coinvolte nel percorso sono:

Rome, Belgrade, Copenhagen, Berlin, Bucharest, Budapest, Bratislava, Athens, Bern, Dublin, Lisbon, Barcelona, Geneva, Helsinki, Kiev, Istanbul, Edinburgh, Ljubljana, Brussels, Frankfurt

Eseguendo la simulazione per 50 volte, lo *scatter plot* degli ottimi trovati è il seguente:

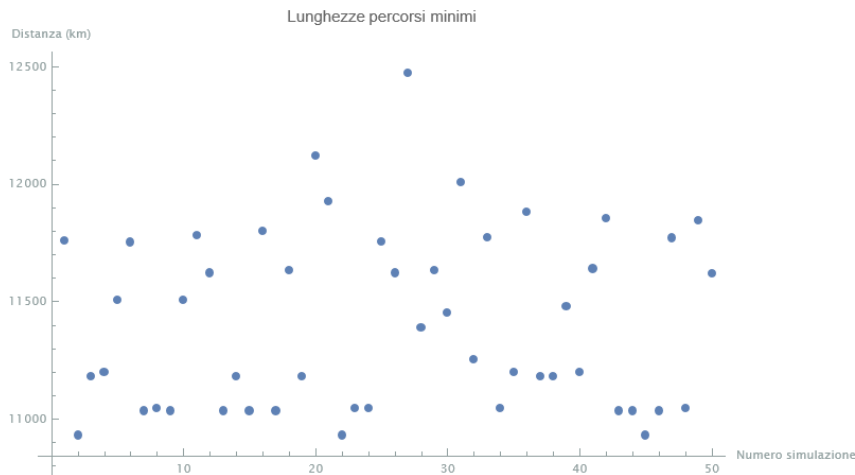


Figura 3.19: Esiti delle 50 iterazioni dell'algoritmo SA, con $n=20$, $ntot=2$, $ntemp=10$, $nsteps=200$, cioè $\chi = 4000$, e raffreddamento geometrico.

La minima distanza fra gli ottimi trovati nella 50 simulazioni è 10930.5 km , e corrisponde al percorso:

Rome, Ljubljana, Bratislava, Budapest, Belgrade, Athens, Istanbul, Bucharest, Kiev, Helsinki, Copenhagen, Berlin, Edinburgh, Dublin, Brussels, Frankfurt, Bern, Geneva, Barcelona, Lisbon

Tale ottimo viene trovato da 3 simulazioni su 50 (6%). Le distanze trovate nelle 50 simulazioni sono comprese nell'intervallo $[10930.5 \text{ km}, 12474.1 \text{ km}]$.

Proviamo ora ad aumentare il parametro χ , sempre a parità di n .

$$n = 20, \text{ } n_{\text{tot}} = 4, \text{ } n_{\text{temp}} = 20, \text{ } n_{\text{steps}} = 10 \cdot n = 200 \quad (3.10)$$

Quindi: $\chi = 16000$. Le città coinvolte nel percorso sono analoghe a quelle viste precedentemente. Effettuando 50 simulazioni, la distribuzione degli ottimi ottenuti è sintetizzata dal grafico a dispersione 3.20.

Evidentemente, la minima distanza fra gli ottimi trovati e il corrispondente percorso (ottimale) sono gli stessi trovati nelle simulazioni con $\chi = 4000$. Il minimo viene individuato da 8 simulazioni su 50 (16%). Dal percorso iniziale di 21828.7 km , tutte le 50 simulazioni trovano percorsi con distanze nell'intervallo $[10930.5 \text{ km}, 12115.2 \text{ km}]$.

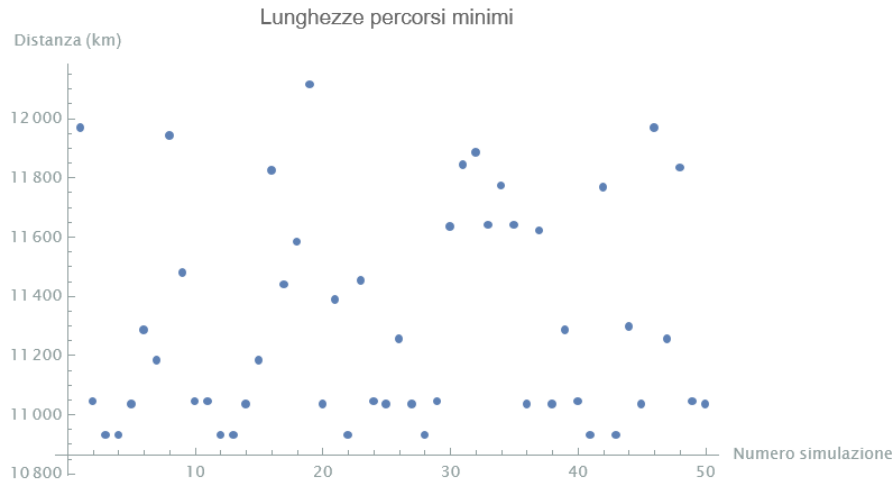


Figura 3.20: Esiti delle 50 iterazioni dell'algoritmo SA, con $n=20$, $n_{\text{tot}}=4$, $n_{\text{temp}}=20$, $n_{\text{steps}}=200$, cioè $\chi = 16000$, e raffreddamento geometrico.

OSSERVAZIONE: Per valori inferiori della complessità computazionale χ le distanze restituite dalle simulazioni sono abbastanza disperse in un intervallo avente per estremo sinistro la migliore approssimazione del minimo. Al contrario, aumentando la complessità computazionale della singola simulazione, aumenta il numero di simulazioni che restituiscono il valore minimo fra tutti gli ottimi trovati, ed i risultati delle singole simulazioni

si addensano maggiormente in corrispondenza dell'estremo sinistro dell'intervallo in cui si trovano, come si riscontra qualitativamente comparando le figure 3.19 e 3.20. Inoltre, per valori di χ maggiori, si riduce l'ampiezza dell'intervallo di dispersione degli esiti.

3.3.3. Simulazioni SA con n elevato e schema di raffreddamento logaritmico

Analogamente a quanto fatto nel paragrafo 3.3.2, poniamo:

$$n = 20, \text{ } n_{tot} = 2, \text{ } n_{temp} = 10, \text{ } n_{steps} = 10 \cdot 20 = 200 \rightarrow \chi = 4000 \quad (3.11)$$

Impostiamo adesso un raffreddamento del parametro di controllo di tipo logaritmico:

$$\beta_n = \beta_0 \cdot \text{Log}(1 + itemp_n), \quad (3.12)$$

ancora con $\beta_0 = 0.1$. Lanciando la simulazione:

Percorso iniziale: Rome, Brussels, Helsinki, Frankfurt, Ljubljana,
Barcelona, Bratislava, Dublin, Berlin, Geneva, Istanbul, Budapest,
Copenhagen, Lisbon, Kiev, Athens, Belgrade, Bern, Bucharest, Edinburgh
Distanza iniziale(km): 28500.7

L'esito è:

Percorso ottimale: Rome, Ljubljana, Bratislava, Budapest, Belgrade, Athens,
Istanbul, Bucharest, Kiev, Helsinki, Copenhagen, Berlin, Frankfurt,
Edinburgh, Dublin, Brussels, Bern, Geneva, Barcelona, Lisbon
Distanza ottimale(km): 11044.7

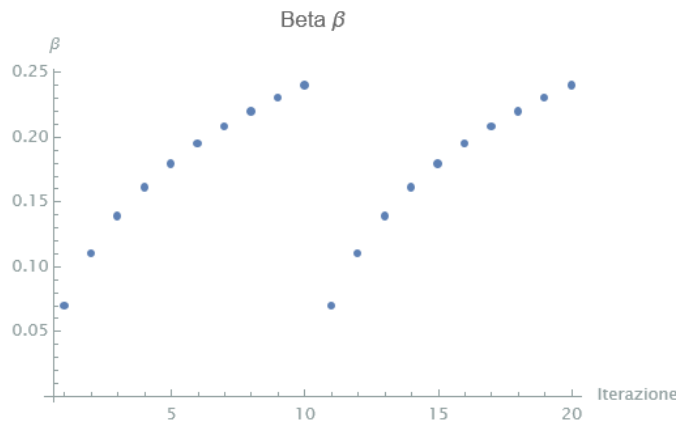


Figura 3.21: Andamento di β nel susseguirsi delle iterazioni.

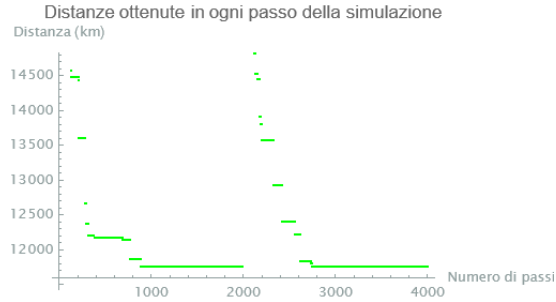


Figura 3.22: Distanze ottenute nei singoli passi della simulazione.

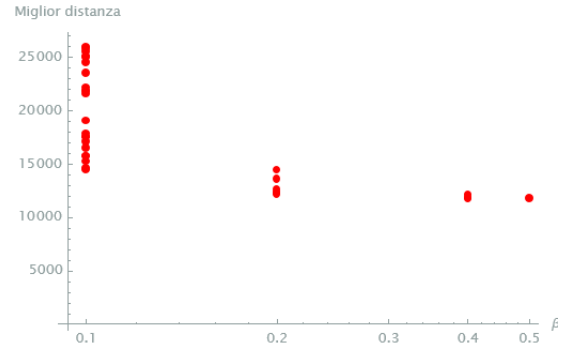


Figura 3.23: Distanze ottimali accettate al variare dei β utilizzati.

Come emerge dalle figure 3.22 e 3.23, il minimo percorso fra quelli ottimali dalle singole simulazioni ha distanza totale di 11044.7 km a fronte dei 28500.7 km del percorso iniziale casualmente generato. Delle due ripetizioni dell'algoritmo SA a partire da stati iniziali diversi, solo la seconda trova tale valore minimo.

Come nel paragrafo 3.3.2, effettuiamo adesso 50 simulazioni, utilizzando dapprima $\chi = 4000$. Come si nota dal grafico a dispersione 3.24, il minimo trovato è ancora di 10930.5 km ; quest'ultimo, insieme con il percorso ad esso corrispondente, sono analoghi a quelli trovati nelle simulazioni a raffreddamento geometrico. Tuttavia, nel caso a raffreddamento logaritmico, tale minimo viene trovato in 6 simulazioni su 50, cioè nel 12% di esse, contro il 6% del caso di raffreddamento geometrico. A pari complessità computazionale, le simulazioni effettuate con raffreddamento logaritmico sembrano portare più volte al percorso minimo rispetto a quelle con raffreddamento geometrico. L'intervallo in cui si collocano gli esiti è $[10930.5 \text{ km}, 12154.2 \text{ km}]$.

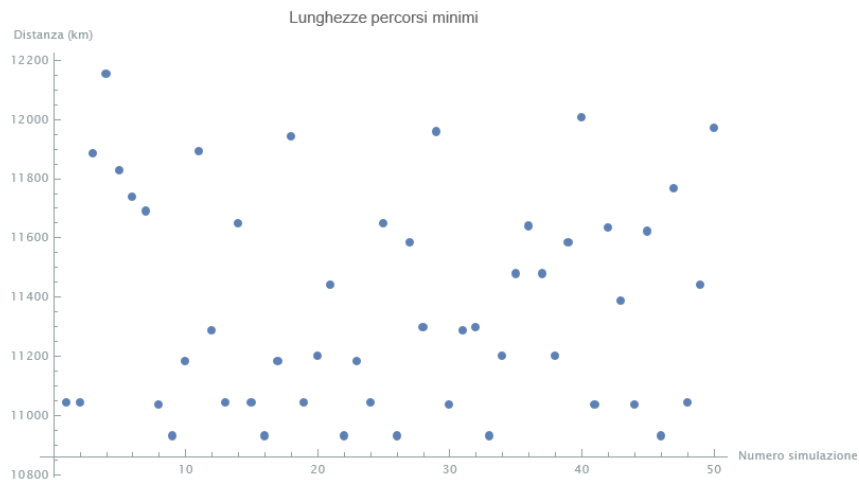


Figura 3.24: Esiti delle 50 iterazioni dell'algoritmo SA, con $n=20$, $ntot=2$, $ntemp=10$, $nsteps=200$, cioè $\chi = 4000$, e raffreddamento logaritmico.

Aumentiamo ora la complessità computazionale, portando χ al valore 16000:

$$n = 20, \text{ } ntot = 4, \text{ } ntemp = 20, \text{ } nsteps = 10 \cdot 20 = 200 \quad (3.13)$$

Eseguendo 50 simulazioni si ricava lo *scatter plot* 3.25. Il minimo valore restituito ed il corrispondente percorso sono analoghi a quelli trovati precedentemente. Tale minimo viene restituito da 11 simulazioni sulle 50 lanciate, cioè nel 22% delle iterazioni, contro il 16% trovato con raffreddamento geometrico, e l'intervallo di comparsa degli esiti è $[10930.5 \text{ km}, 11922.9 \text{ km}]$.

OSSERVAZIONE: Come notato nella sezione precedente, anche nel caso di raffreddamento logaritmico gli esiti si addensano verso l'estremo sinistro dell'intervallo all'aumentare di χ ; inoltre, aumentando lo stesso parametro, il minimo viene ottenuto un numero di volte percentualmente maggiore, e l'intervallo in cui sono distribuiti gli esiti, a parità di numero di simulazioni, si restringe.

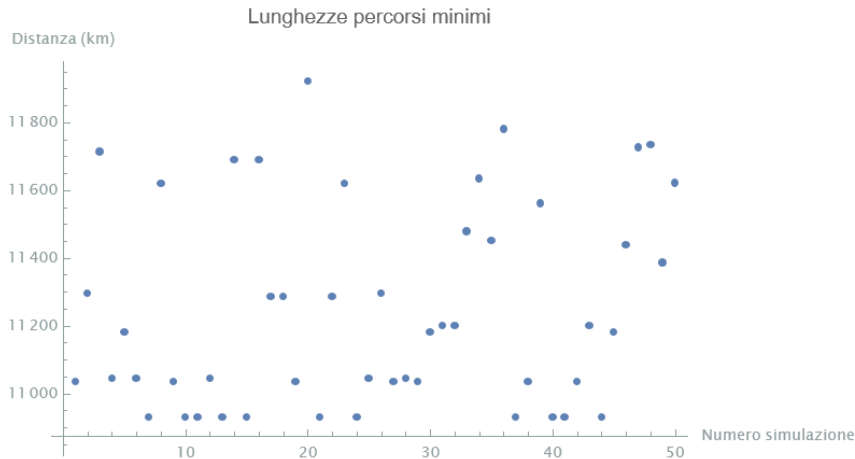


Figura 3.25: Esiti delle 50 iterazioni dell'algoritmo SA, con $n=20$, $ntot=4$, $ntemp=20$, $nsteps=200$, cioè $\chi = 16000$, e raffreddamento logaritmico.

3.3.4. Conclusioni sul metodo di raffreddamento

Consideriamo una visione d'insieme sui quattro casi studiati, al variare del parametro χ e del metodo di raffreddamento.

È già stato notato che, all'aumentare della complessità computazionale χ , sale la percentuale di simulazioni che restituiscono il minimo, fissato un qualsiasi metodo di raffreddamento, e diminuisce la variabilità degli esiti.

Compariamo ora, a parità di χ , come si comportano *insiemi di simulazioni* condotte con ciascuno dei due tipi di raffreddamento.

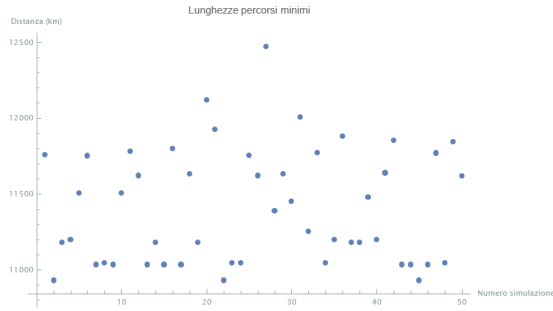


Figura 3.26: $\chi = 4000$, raffreddamento geometrico.

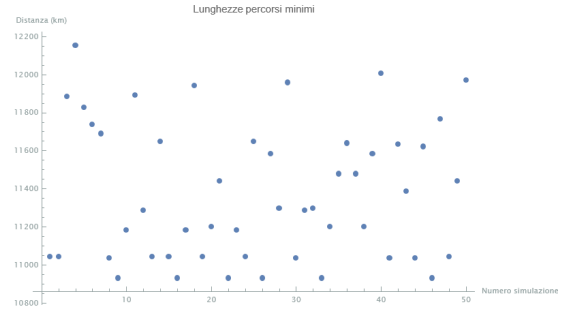


Figura 3.27: $\chi = 4000$, raffreddamento logaritmico.

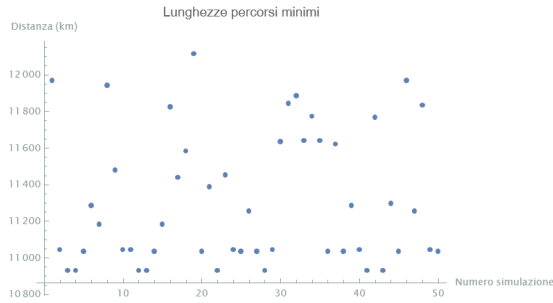


Figura 3.28: $\chi = 16000$, raffreddamento geometrico.

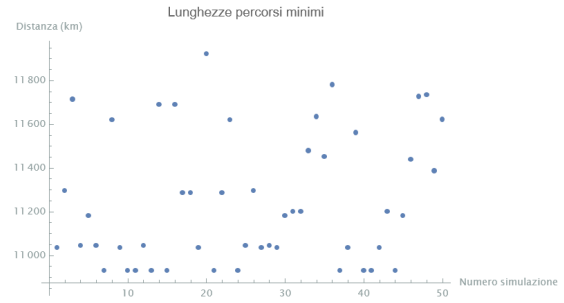


Figura 3.29: $\chi = 16000$, raffreddamento logaritmico.

Emergono qualitativamente tre punti:

- Gli intervalli in cui sono distribuiti gli esiti delle simulazioni sono comparabili nei due casi di raffreddamento logaritmico e geometrico. Nonostante le simulazioni a raffreddamento logaritmico presentino intervalli leggermente meno ampi (con estremo destro minore per entrambi i valori di χ), tali differenze sono ragionevolmente imputabili a fluttuazioni casuali e circostanziali, a cui non sono attribuibili considerazioni essenziali sui metodi di raffreddamento.
- Le simulazioni con raffreddamento logaritmico appaiono più disperse di quelle con raffreddamento geometrico. Gli esiti delle prime si collocano quasi uniformemente nell'intervallo di distanze (con leggera prevalenza verso distanze minori) per $\chi = 4000$, e si concentrano maggiormente verso l'estremo sinistro dell'intervallo per $\chi = 16000$. Nel caso di raffreddamento geometrico, gli esiti risultano più addensati verso l'estremo sinistro, sia per $\chi = 4000$ che per $\chi = 16000$.
- Nonostante la maggiore variabilità delle simulazioni logaritmiche rispetto a quelle geometriche, le prime riescono a trovare più frequentemente i percorsi minimi

ottimali, o comunque la loro migliore stima.

Per spiegare queste caratteristiche è utile ricordare gli andamenti di β per i due tipi di raffreddamento.

Ricordiamo che:

- Un valore di β basso (temperatura alta) implica che la corrispondente catena di Markov omogenea sarà caratterizzata da alta probabilità di accettazione di scelte peggiorative. La simulazione esplora quindi in modo quasi-casuale lo spazio delle configurazioni.
- Al contrario, un alto valore di β (temperatura bassa) comporta un quasi sicuro rifiuto di scelte peggiorative nella DTMC.

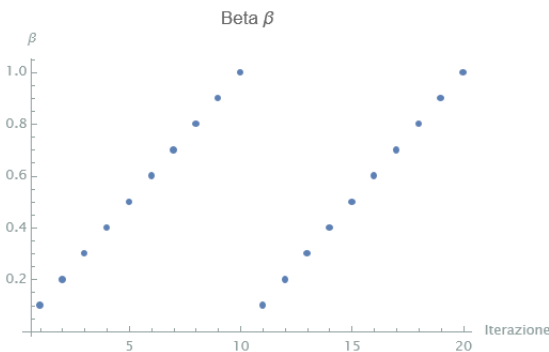


Figura 3.30: Raffreddamento geometrico.

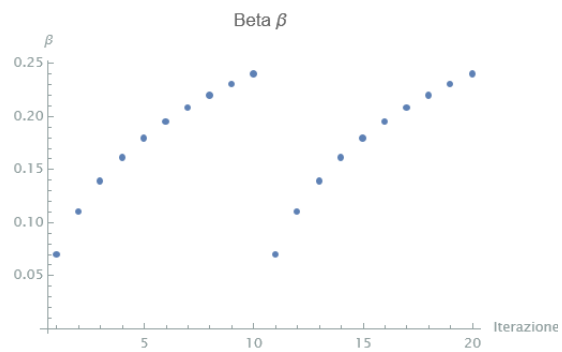


Figura 3.31: Raffreddamento logaritmico.

Abbassando il parametro di controllo, quindi, la simulazione si "congela" con buona probabilità in corrispondenza di un minimo, locale o globale.

Nel raffreddamento logaritmico, l'abbassamento di temperatura è più lento e graduale al passare delle iterazioni. Questo comporta che l'algoritmo impiega un numero maggiore di passi a temperature maggiori, con conseguente maggiore esplorazione pseudo-casuale dello spazio delle configurazioni, e minor numero di passi nel "congelamento" verso un minimo. Questa caratteristica, se da un lato giustifica la maggiore dispersione degli esiti delle simulazioni logaritmiche rispetto a quelle geometriche, rende anche conto della maggiore frequenza di comparsa della distanza ottimale nelle prime. Una più estensiva esplorazione dello spazio di stato permette infatti di sfuggire con maggiore probabilità a minimi locali non assoluti. Al contrario, le simulazioni con raffreddamento geometrico sono caratterizzate da un più repentino trend discendente del parametro di controllo; l'esplorazione dello spazio delle configurazioni è quindi meno estensiva, e la simulazione si avvicina più rapidamente verso un minimo, locale o globale.

4 | Conclusioni

Nei capitoli precedenti è stato delineato un percorso teorico che, a partire dalla meccanica statistica e dall'uso che in essa si fa dei metodi MC, ha permesso di esplorare due algoritmi di approssimazione standardizzati (Metropolis e SA) per la risoluzione dell'archetipo dei problemi di ottimizzazione combinatoria: il TSP.

L'algoritmo di Metropolis, iterato fino a convergenza, ha dimostrato di essere efficiente nella determinazione del minimo globale della distanza totale per itinerari a basso numero di città. Aumentando tale numero, è stata riscontrata la comparsa di minimi locali, che rendono tale algoritmo inaffidabile per l'ottimizzazione: esso si arena facilmente in minimi locali non globali, e risulta comunque troppo basilare per la risoluzione di un problema complesso a molte città. In entrambi questi casi, si è inizialmente imposto il vincolo di mantenere fisso il punto di partenza.

Eliminando questo vincolo, è emerso come il semplice algoritmo di Metropolis, iterato fino a convergenza, sia talvolta in grado di individuare con buona probabilità quali devono essere le città all'inizio e alla fine del percorso per riuscire a minimizzare la distanza, anche in presenza di minimi locali, o quantomeno per ottenere un minimo locale.

Per innalzare il livello algoritmico, si è poi proceduto implementando il *Simulated Annealing*, iterando cioè simulazioni di tipo Metropolis abbassando gradualmente il parametro di controllo. In questo modo, anche problemi con un numero più elevato di città sono stati affrontati basandosi sui risultati di molteplici esecuzioni dell'algoritmo SA, ed estraendo i minimi percorsi ottenuti nelle suddette simulazioni come migliori stime del minimo globale della distanza totale.

Infine, nell'ambito delle simulazioni SA, si sono evidenziate le diverse specificità conseguenti all'utilizzo di uno dei due schemi di raffreddamento introdotti. In particolare, per via dell'andamento funzionale del parametro di controllo, è emerso come il raffreddamento logaritmico porti la simulazione al minimo globale con maggiore probabilità rispetto al raffreddamento geometrico; tuttavia, lo stesso raffreddamento logaritmico porta ad una maggiore dispersione negli esiti delle simulazioni, a causa della più lenta discesa del parametro di controllo.

Bibliografia

- [1] M. E. J. Newman and G. T. Barkema. Monte Carlo Methods in Statistical Physics. Oxford University Press, 1999.
- [2] P. J. M. van Laarhoven and E. H. L. Aarts. Simulated Annealing: Theory and Applications. Springer, 1987.
- [3] W. L. Hosch. P versus NP problem, 2024. URL <https://www.britannica.com/science/P-versus-NP-problem#ref1119246>.
- [4] P. Biscari. Dispense del corso "Physics of complex systems".
- [5] F. Ciccacci. Fondamenti di Fisica Atomica e Quantistica. EdiSES Università, 2019.
- [6] R. J. Wilson. Introduction to Graph Theory. Prentice Hall, 1998.
- [7] K. Huang. Statistical Mechanics, Wiley, 2008.
- [8] D. A. Levin, Y. Peres. Markov Chains and Mixing Times. American Mathematical Society, 2017.
- [9] F. Schabl. Statistical Mechanics. Springer, 2006.
- [10] A. Toigo. Dispense del corso "Statistica".
- [11] F. Cowell. Measuring Inequality. Oxford University Press, 2011.
- [12] OECD (2011), "Income inequality", in Society at a Glance 2011: OECD Social Indicators, OECD Publishing, Paris.

A | Spazio delle fasi

Lo spazio delle fasi (o spazio delle configurazioni) Γ di un sistema a N particelle è definito in meccanica hamiltoniana come l'iperspazio i cui $6N$ assi sono le coordinate e i momenti generalizzati dei costituenti il sistema. L'unità di misura dei volumi in Γ si determina con l'analisi dimensionale seguente:

$$[\Gamma] = \left[\prod_{i=1}^N p_i q_i \right] = [S]^{3N} = [J \cdot s]^{3N} \quad (\text{A.1})$$

dove " J " e " s " indicano le unità di misura SI dell'energia (joule) e del tempo (secondo), e S è il funzionale azione, definito come l'integrale temporale della lagrangiana:

$$S := \int_{t_1}^{t_2} L(\mathbf{q}(\mathbf{t}), \mathbf{p}(\mathbf{t}), t) dt \quad (\text{A.2})$$

Ogni punto di Γ identifica (in meccanica statistica classica) un microstato del sistema. Passando ad una visione quantistica, emerge immediatamente l'esigenza di quantizzare lo spazio delle fasi, ossia di individuarne la cella elementare, il cui volume corrisponde ad un singolo stato quantistico. Ricordiamo che la discretizzazione in celle elementari è pur sempre un concetto di natura semiclassica e presenta dunque alcuni limiti. Può essere tuttavia utile per un primo approccio alla meccanica statistica quantistica.

Dal principio di indeterminazione posizione-momento:

$$\Delta x \Delta p \geq \hbar \quad (\text{A.3})$$

Il volume minimo permesso dalla (A.3) per uno stato quantistico **di singola particella** rispetta:

$$\Delta x \Delta y \Delta z \Delta p_x \Delta p_y \Delta p_z \geq \hbar^3 \quad (\text{A.4})$$

Si dimostra facilmente (applicando per esempio condizioni al contorno cicliche nel problema della buca di potenziale infinita rettangolare 3D) che il volume elementare in Γ per

un sistema a N particelle vale:

$$\delta\Gamma = h^{3N} \quad (\text{A.5})$$

Si consideri per semplicità un gas perfetto. Il volume totale dello spazio delle fasi è dato da:

$$V_\Gamma = V^N \cdot p_{max}^{3N} \quad (\text{A.6})$$

dove p_{max} è il massimo valore di quantità di moto delle particelle (atomi o molecole) nel gas, e V è il volume fisico da esse occupato. Si può stimare il momento delle particelle in modo grossolano:

$$\bar{E} \sim k_B T \rightarrow \bar{p} \sim \sqrt{2mk_B T} \quad (\text{A.7})$$

Si calcola ora il numero approssimato di microstati η dividendo il volume dello spazio delle fasi per il volume di un singolo stato:

$$\eta(E) \sim \frac{V_\Gamma}{h^{3N}} \sim \frac{V^N \cdot (\sqrt{2mk_B T})^{3N}}{h^{3N}} \quad (\text{A.8})$$

Come specificato prima, il ragionamento fatto è evidentemente tutt'altro che rigoroso, e fa uso di concetti al più semiclassici, come quello di spazio delle fasi. Un conto più preciso, che fa uso di metodi statistici quantistici più sofisticati, è mostrato in [9] (*F. Schwabl. Statistical Mechanics. Springer, 2006.*), e porta al risultato seguente:

$$\eta(E) = \frac{V^N (2\pi m E)^{\frac{3N}{2}}}{h^{3N} N! (\frac{3N}{2})!} \quad (\text{A.9})$$

Nel limite di N grande, vale l'espressione approssimata:

$$\eta(E) \sim \left(\frac{V}{N}\right)^N \left(\frac{4\pi m E}{3h^2 N}\right)^{\frac{3N}{2}} e^{\frac{5N}{2}} \frac{1}{E} \frac{3N}{2} \quad (\text{A.10})$$

Per la stima, utilizziamo la formula classica (A.8). Nel caso di un gas perfetto a $T = 300 \text{ K}$, massa dei costituenti $m = 10^{-27} \text{ kg}$, numero di costituenti $N = N_A$ e volume $V = 1 \text{ m}^3$ si otterrebbe:

$$\log(\eta) \sim 4 \cdot 10^{25} \quad (\text{A.11})$$

Il numero classico di microstati in Γ è elevatissimo! Si assuma un tasso di cambiamento di stato di 10^{12} cambiamenti al secondo. In un'ora, ad esempio, il sistema esplorerebbe appena

$$10^{12} \text{ s}^{-1} \cdot 3600 \text{ s} = 3.6 \cdot 10^{15} \text{ stati} \quad (\text{A.12})$$

che è un numero enormemente minore di η .

B | Passeggiate aleatorie su grafi

Consideriamo un modo alternativo per introdurre le catene di Markov a tempo discreto (DTMC), che sfrutta il concetto di grafo.

Definizione B.1. *Un grafo semplice G consiste in un insieme finito e non vuoto $V(G)$ di elementi detti **vertici** (o nodi) e in un insieme finito $E(G)$ di coppie distinte di elementi di $V(G)$ detti **spigoli** (o archi). $V(G)$ è detto **insieme dei vertici**, $E(G)$ è detto **insieme degli spigoli**.*

Si dice che lo spigolo (μ, ν) *connette* i vertici μ e ν ; tale spigolo si indica come $\mu\nu$.

Definizione B.2. *Il **grado** ($\deg(\mu)$) di un vertice μ di un grafo è il numero di spigoli che hanno tale vertice come componente della coppia di vertici.*

Una **passeggiata** sul grafo è una sequenza ordinata di spigoli, tali che l'*endpoint* del precedente coincida con lo *startpoint* del successivo. Un **percorso** è una passeggiata in cui nessun vertice appare per più di una volta. Si arriva quindi ad una introduzione alternativa del concetto di DTMC:

Definizione B.3. *Una **passeggiata aleatoria** su un grafo $(V(G), E(G))$ è una DTMC la cui matrice stocastica ha elementi $P_{\mu\nu}$ non nulli se e solo se i vertici μ, ν sono connessi nel grafo.*

Gli stati del sistema sono quindi visti in quest'ottica come i vertici di un grafo. La passeggiata aleatoria si definisce **semplice** se **per ogni** vertice μ vale:

$$\forall \nu : P_{\mu\nu} \neq 0, P_{\mu\nu} = \frac{1}{\deg(\mu)} \quad (\text{B.1})$$

Definizione B.4. *Un grafo è **connesso** se per ogni coppia di stati (μ, ν) esiste un percorso che li collega.*

Risulta quindi chiaro che:

Proposizione B.1. *Una passeggiata aleatoria su un grafo (quindi una DTMC) è irriducibile se e solo se il grafo è connesso.*

Ringraziamenti

La scrittura di una tesi rappresenta un punto di arrivo e di partenza, la conclusione di un percorso e l'inizio di un altro. La vita è cambiamento, fluire di esperienze, persone, luoghi. Sono stati tre anni intensi. All'inizio spaesanti, difficili. Una nuova città, tanti timori e dubbi. Ma presto ho conosciuto persone solari e uniche, che mi hanno reso felice, e con cui sono cresciuto e dalle quali/con le quali ho imparato molto.

Ringrazio per questo tutti gli amici del Poli, quelli con cui si è formato un legame più forte, punti di riferimento ormai della mia vita, ma anche gli altri, perchè anche una risata, uno scambio di esperienze o una chiacchierata ogni tanto sono piccoli momenti di gioia.

Ringrazio i miei amici "di giù", compagni di vita, persone speciali. Anche dopo mesi, rivedersi, parlare, ridere insieme sono azioni ancora naturali, spontanee e leggere ma importanti, come lo erano quando ogni mattina ci vedevamo al liceo.

Ringrazio la mia famiglia, che ha reso tutto questo possibile, che mai mi ha fatto mancare supporto, amore e fiducia. Presenze costanti ormai da poco più di vent'anni, àncore a cui devo tutto, che mi ispirano continuamente e grazie alle quali sto diventando un uomo.

Ringrazio tutti i miei prof del liceo, per essere stati figure trainanti nella mia adolescenza, e per aver acceso in me la passione per l'apprendere, il fascino della conoscenza come strumento di crescita e di elevazione dell'essere umano.

